

Language Models

Technical Notes

Rodrigo Kang

Table of contents

1	Language as a Representation Problem	1
2	Bag-of-Words	2
2.1	Tokenisation	3
2.2	Vocabulary	3
2.3	Vector Representation	4
2.4	Algorithm	6
2.5	Python Implementation	7
2.6	Limitations of Bag-of-Words	8
2.6.1	Loss of Word Order	8
2.6.2	Lack of Semantic Structure	9
2.6.3	High Dimensionality and Sparsity	10
3	Dense Vector Embeddings	11
3.1	Similarity in the Embedding Space	12
3.2	Learning the Representation	13
4	Context Matters	14
4.1	From Static to Contextual Representations	16
5	Attention	16
5.1	Attention Weights	18
5.2	From Attention to Transformers	19
6	The Transformer	19
6.1	Encoder-Decoder Architecture	20
6.2	Self-Attention as Contextualisation	21
7	Representation and Generative Models	22
7.1	Encoder-Only Models	23
7.1.1	Masked Language Modelling	24
7.2	Decoder-Only Models	24
7.3	Two Different Objectives	25

8	Pretraining and Fine-Tuning	26
8.1	Pretraining	26
8.2	Fine-Tuning	27
8.3	Post-Training	28
9	Using a Pretrained Language Model	29
9.1	Generation Pipeline	30
9.2	Python Implementation	31
10	Tokenisation	32
10.1	Token IDs	32
10.2	Encoding and Decoding	33
10.3	Tokenisation Granularity	34
10.3.1	Word Tokens	34
10.3.2	Subword Tokens	35
10.3.3	Character Tokens	36
10.3.4	Byte Tokens	36
10.4	The Granularity Trade-off	37
10.5	Tokenizer Design	37
10.5.1	Tokenisation Method	38
10.5.2	Vocabulary Size	38
10.5.3	Special Tokens	39
10.5.4	Capitalisation and Whitespace	39
10.5.5	Training-Data Domain	40
10.6	Python Implementation	41
11	Token Embeddings	41
11.1	The Embedding Matrix	42
11.2	Contextualised Token Embeddings	43
11.2.1	Output Representation	45
11.3	Python Implementation	45
12	Text Embeddings	46
12.1	Pooling Token Representations	47
12.2	Python Implementation	48
13	Word2vec	48
13.1	Skip-Gram	49
13.2	Positive and Negative Examples	50
13.3	Learning Word Embeddings	50
13.4	Negative Sampling	51
13.5	Algorithm	53
13.6	Gradient Derivation	55
13.6.1	Gradient with Respect to the Centre Embedding	56
13.6.2	Gradient with Respect to the Context Embeddings	56
13.7	Python Implementation	57
14	Embeddings Beyond Language	58

15	Autoregressive Text Generation	59
15.1	The Forward Pass	60
15.2	Decoding the Next Token	62
15.2.1	Greedy Decoding	63
15.2.2	Sampling	63
16	Parallel Token Processing	64
16.1	Context Length	65
17	Key-Value Cache	66
18	Transformer Blocks	67
18.1	Feedforward Neural Network	68
19	Self-Attention	70
19.1	Queries, Keys, and Values	71
19.2	Scaled Dot-Product Attention	73
19.3	Causal Masking	75
19.4	Algorithm	76
19.5	Multi-Head Attention	78
19.6	Multi-Query and Grouped-Query Attention	79
19.6.1	Multi-Query Attention	80
19.6.2	Grouped-Query Attention	80
19.7	Efficient Attention	81
19.8	Residual Connections and Normalisation	82
19.8.1	Residual Connections	82
19.8.2	Normalisation	83
19.8.3	Pre-Normalisation	84
20	Positional Information	84
20.1	Positional Embeddings	85
20.2	Rotary Position Embeddings	86
21	The Transformer Decoder	88
22	Inspecting a Transformer Model	90
22.1	Model Architecture	90
22.1.1	Gated Feedforward Networks	92

1 Language as a Representation Problem

Human language is naturally expressed through discrete symbols such as words, subwords, and characters. A machine-learning model, however, does not operate directly on these symbols. Its inputs must ultimately be represented numerically.

This creates a fundamental problem in language modelling:

How can language be represented numerically while preserving the information required to perform a particular task?

Consider, for example, the following two sentences:

The cat is sleeping.

The dog is sleeping.

To a human reader, the sentences are clearly related. They have almost the same syntactic structure and describe similar situations, differing primarily in the subject. A computational model, however, cannot directly perform mathematical operations on the words *cat*, *dog*, or *sleeping*. We first need a mapping from language to a numerical representation.

Let a text sequence be denoted by

$$S = (w_1, w_2, \dots, w_N), \quad (1)$$

where each w_i represents a linguistic unit, such as a word or token. We seek a representation function

$$\varphi : \mathcal{S} \rightarrow \mathbb{R}^d, \quad (2)$$

which maps a sequence S from the space of possible text sequences $\mathcal{S}$ to a numerical vector

$$\mathbf{x} = \varphi(S) = (x_1, x_2, \dots, x_d)^\top. \quad (3)$$

Once language has been represented numerically, the resulting vector can be used as the input to computational methods for tasks such as classification, retrieval, clustering, or generation.

The difficulty is that there is no unique numerical representation of language. Different mappings preserve different kinds of information. For example, a representation based only on word counts can preserve information about which words occur in a document and how frequently they occur, but it largely discards word order and context. More sophisticated representations attempt to preserve semantic or contextual relationships between linguistic units.

The representation therefore determines, to a significant extent, what information is available to the model. This observation provides a useful way to understand the historical development of language models. Many important advances in Natural Language Processing can be viewed as increasingly expressive ways of representing language:

word counts $\longrightarrow$ static embeddings $\longrightarrow$ contextual representations.

We begin with one of the simplest numerical representations of text: the **bag-of-words model**.

2 Bag-of-Words

The representation problem becomes concrete as soon as we try to perform a computational task with text. Suppose, for example, that we have collected three short customer reviews:

The product is good.

The product is very good.

The service is bad.

Before these reviews can be used by a machine-learning algorithm, we need to transform them into numerical representations. One of the simplest approaches is the **bag-of-words model**.

Bag-of-words represents a document by the words that occur in it and their frequencies. The central idea is deliberately simple: construct a vocabulary from the available documents and associate each vocabulary term with one component of a vector.

2.1 Tokenisation

Let the collection of documents, or **corpus**, be

$$\mathcal{D} = \{D_1, D_2, \dots, D_M\}, \quad (4)$$

where M is the number of documents.

For our example,

$$\begin{aligned} D_1 &= \text{"the product is good"}, \\ D_2 &= \text{"the product is very good"}, \\ D_3 &= \text{"the service is bad"}. \end{aligned}$$

The first step is **tokenisation**, which divides each document into smaller linguistic units called tokens. For now, we use the simplest possible tokenizer: convert the text to lowercase, remove punctuation, and split on whitespace.

This gives

$$\begin{aligned}
T(D_1) &= (\text{the, product, is, good}), \\
T(D_2) &= (\text{the, product, is, very, good}), \\
T(D_3) &= (\text{the, service, is, bad}).
\end{aligned}$$

This simple procedure is sufficient for the present example, but tokenisation is itself a nontrivial problem. Modern language models commonly operate on subword or byte-level tokens rather than simply splitting text on whitespace. Tokenisation will therefore be studied separately later.

2.2 Vocabulary

Once the documents have been tokenised, we construct a **vocabulary** containing the unique tokens appearing in the corpus.

For the example above,

$$\mathcal{V} = \{\text{the, product, is, good, very, service, bad}\}.$$

The vocabulary therefore contains

$$|\mathcal{V}| = 7$$

distinct terms.

Let

$$\mathcal{V} = (v_1, v_2, \dots, v_d), \tag{5}$$

where

$$d = |\mathcal{V}|.$$

The ordering of the vocabulary is arbitrary, but once an ordering has been chosen it must remain fixed because each term determines the meaning of one coordinate of the resulting vectors.

For our chosen ordering,

$$\begin{aligned}
v_1 &= \text{the}, & v_2 &= \text{product}, & v_3 &= \text{is}, \\
v_4 &= \text{good}, & v_5 &= \text{very}, & v_6 &= \text{service}, & v_7 &= \text{bad}.
\end{aligned}$$

2.3 Vector Representation

We can now define the bag-of-words representation of a document D as

$$\varphi(D) = \mathbf{x} = (x_1, x_2, \dots, x_d)^\top, \quad (6)$$

where each component is the number of times the corresponding vocabulary term occurs in the document:

$$x_j = \text{count}(v_j, D), \quad j = 1, \dots, d. \quad (7)$$

Consider the first document,

$$D_1 = \text{"the product is good"}.$$

Using the vocabulary defined above, the individual counts are

$$\begin{aligned} \text{count}(\text{the}, D_1) &= 1, \\ \text{count}(\text{product}, D_1) &= 1, \\ \text{count}(\text{is}, D_1) &= 1, \\ \text{count}(\text{good}, D_1) &= 1, \\ \text{count}(\text{very}, D_1) &= 0, \\ \text{count}(\text{service}, D_1) &= 0, \\ \text{count}(\text{bad}, D_1) &= 0. \end{aligned}$$

Therefore,

$$\varphi(D_1) = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}.$$

Applying the same procedure to the remaining documents gives

$$\varphi(D_2) = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 0 \\ 0 \end{pmatrix}, \quad \varphi(D_3) = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 0 \\ 1 \\ 1 \end{pmatrix}.$$

The complete corpus can therefore be represented by the **document-term matrix**

$$X = \begin{pmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 & 1 \end{pmatrix}, \quad (8)$$

where each row represents a document and each column represents a term in the vocabulary.

More generally,

$$X \in \mathbb{R}^{M \times d},$$

with elements

$$X_{ij} = \text{count}(v_j, D_i). \quad (9)$$

Bag-of-words has therefore transformed a collection of variable-length symbolic sequences into fixed-dimensional numerical vectors that can be processed by standard statistical and machine-learning algorithms.

2.4 Algorithm

The construction can be expressed as the following procedure:

Input: a corpus $\mathcal{D} = \{D_1, \dots, D_M\}$.

Output: a vocabulary $\mathcal{V}$ and document-term matrix X .

1. Tokenise every document in the corpus.
2. Collect the unique tokens across all documents to construct the vocabulary $\mathcal{V}$.
3. Assign each vocabulary term v_j to a column j .
4. Initialise $X \in \mathbb{R}^{M \times d}$ with zeros.
5. For each document D_i :

1. For each token in D_i :
 1. Find the vocabulary index j associated with the token.
 2. Increment X_{ij} by one.
6. Return $\mathcal{V}$ and X .

The computational representation is now explicit: text has become a matrix of numerical features. However, this simplicity comes at a cost. The representation records **which words occur and how often they occur**, but not the order in which they occur.

For example, the sentences

dog bites man

and

man bites dog

contain exactly the same tokens with exactly the same frequencies. Under a unigram bag-of-words representation, both therefore produce the same vector:

$$\varphi(\text{"dog bites man"}) = \varphi(\text{"man bites dog"}).$$

Their meanings are clearly different, but that difference has been lost during the mapping from language to the numerical representation.

This illustrates a fundamental trade-off. Bag-of-words provides a simple and useful numerical representation of text, but it does not model word order, context, or semantic relationships between words. These limitations motivate the development of richer language representations.

2.5 Python Implementation

The bag-of-words representation can be implemented directly from the procedure described above. To make the construction explicit, the first implementation uses only Python's built-in data structures rather than a specialised Natural Language Processing or machine-learning library.

The implementation first tokenises each document, constructs the vocabulary from the unique tokens in the corpus, and then counts the occurrences of each vocabulary term in every document. The resulting vectors are collected into the document-term matrix X .

The complete implementation is available in `python/bag_of_words.py`.

For the corpus used in the previous example,

$$\begin{aligned}
D_1 &= \text{"the product is good"}, \\
D_2 &= \text{"the product is very good"}, \\
D_3 &= \text{"the service is bad"},
\end{aligned}$$

the implementation produces the vocabulary

$$\mathcal{V} = (\text{the, product, is, good, very, service, bad}),$$

and the document-term matrix

$$X = \begin{pmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 & 1 \end{pmatrix}.$$

The numerical result therefore reproduces the representation obtained analytically.

2.6 Limitations of Bag-of-Words

Bag-of-words provides a simple mapping from text to numerical vectors, but the simplicity of this representation also imposes important limitations. The resulting vectors describe the occurrence of vocabulary terms without explicitly representing the linguistic relationships between them.

2.6.1 Loss of Word Order

The bag-of-words representation depends only on the number of times each vocabulary term occurs in a document. Consequently, any two documents containing the same words with the same frequencies are represented by the same vector, regardless of their ordering.

Consider the sentences

dog bites man

and

man bites dog.

Using the vocabulary

$$\mathcal{V} = (\text{dog}, \text{bites}, \text{man}),$$

both sentences are represented by

$$\mathbf{x} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}.$$

Therefore,

$$\varphi(\text{"dog bites man"}) = \varphi(\text{"man bites dog"}),$$

even though the two sentences describe different situations.

The mapping φ is therefore not one-to-one: distinct text sequences can be mapped to exactly the same numerical representation. Once this information has been discarded by the representation, a model receiving only the bag-of-words vector cannot reconstruct the original word order.

2.6.2 Lack of Semantic Structure

A second limitation is that the coordinates of a bag-of-words vector correspond to vocabulary terms, but the representation itself does not encode semantic relationships between those terms.

Suppose the vocabulary is

$$\mathcal{V} = (\text{cat}, \text{dog}, \text{car}).$$

The individual words are represented by the basis vectors

$$\mathbf{e}_{\text{cat}} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad \mathbf{e}_{\text{dog}} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \quad \mathbf{e}_{\text{car}} = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}.$$

Although *cat* and *dog* are semantically more closely related than *cat* and *car*, their geometric relationships in this representation are identical.

For example,

$$\mathbf{e}_{\text{cat}}^\top \mathbf{e}_{\text{dog}} = 0,$$

and

$$\mathbf{e}_{\text{cat}}^\top \mathbf{e}_{\text{car}} = 0.$$

More generally, for two different vocabulary terms v_i and v_j ,

$$\mathbf{e}_i^\top \mathbf{e}_j = 0, \quad i \neq j. \quad (10)$$

Thus, different vocabulary terms occupy mutually orthogonal directions in the representation space. From the geometry of the representation alone, *cat* is no more similar to *dog* than it is to *car*.

2.6.3 High Dimensionality and Sparsity

The dimensionality of a bag-of-words vector is determined by the size of the vocabulary:

$$\mathbf{x} \in \mathbb{R}^{|\mathcal{V}|}.$$

For a large corpus, $|\mathcal{V}|$ may be very large. At the same time, an individual document generally contains only a small fraction of all vocabulary terms.

If a document contains K distinct vocabulary terms, then its bag-of-words vector contains at most K nonzero components. Its proportion of zero-valued components is therefore

$$1 - \frac{K}{|\mathcal{V}|}. \quad (11)$$

When

$$K \ll |\mathcal{V}|,$$

most components are zero and the representation is **sparse**.

For example, if a vocabulary contains 50 000 terms and a document contains only 100 distinct vocabulary terms, then the proportion of zero components is approximately

$$1 - \frac{100}{50\,000} = 0.998.$$

Thus, approximately 99.8% of the vector contains zeros.

The problem is therefore not merely that bag-of-words produces numerical representations, but that these representations are typically high-dimensional and sparse while providing little explicit information about semantic relationships between words.

These limitations motivate a different approach: rather than assigning each word an independent coordinate in a high-dimensional vocabulary space, words can be represented by learned vectors in a lower-dimensional continuous space. These representations are known as **dense vector embeddings**.

3 Dense Vector Embeddings

The bag-of-words representation assigns each vocabulary term to an independent coordinate of the representation space. As a consequence, different words are represented by orthogonal directions, regardless of their semantic relationship.

A more useful representation would have a different geometric structure. If two words are semantically related, their numerical representations should reflect that relationship.

Consider again the words *cat*, *dog*, and *car*. Rather than representing them as basis vectors in a vocabulary space,

$$\mathbf{e}_{\text{cat}} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad \mathbf{e}_{\text{dog}} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \quad \mathbf{e}_{\text{car}} = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix},$$

we would like to construct representations such that the geometry of the vector space reflects useful relationships between the words.

For example, in a two-dimensional representation space we might obtain

$$\mathbf{e}_{\text{cat}} = \begin{pmatrix} 0.8 \\ 0.7 \end{pmatrix}, \quad \mathbf{e}_{\text{dog}} = \begin{pmatrix} 0.9 \\ 0.6 \end{pmatrix}, \quad \mathbf{e}_{\text{car}} = \begin{pmatrix} -0.7 \\ 0.2 \end{pmatrix}.$$

These particular values are only illustrative. The important difference is that the coordinates are no longer predetermined by the identity of the words. Instead, each word is represented by a point in a continuous vector space.

Such a representation is called an **embedding**.

More generally, let the vocabulary be

$$\mathcal{V} = \{v_1, v_2, \dots, v_{|\mathcal{V}|}\}.$$

An embedding can be viewed as a mapping

$$E : \mathcal{V} \rightarrow \mathbb{R}^d, \quad (12)$$

where

$$d \ll |\mathcal{V}|.$$

Each vocabulary term v_i is therefore associated with an embedding vector

$$\mathbf{e}_i = E(v_i) \in \mathbb{R}^d. \quad (13)$$

Collecting all vocabulary embeddings produces an **embedding matrix**

$$\mathbf{E} = \begin{pmatrix} \mathbf{e}_1^\top \\ \mathbf{e}_2^\top \\ \vdots \\ \mathbf{e}_{|\mathcal{V}|}^\top \end{pmatrix} \in \mathbb{R}^{|\mathcal{V}| \times d}. \quad (14)$$

Unlike the sparse vectors produced by bag-of-words, the vectors $\mathbf{e}_i$ generally contain nonzero real-valued components. They are therefore referred to as **dense vector representations**.

3.1 Similarity in the Embedding Space

Once words are represented as vectors in a continuous space, their relationships can be studied geometrically.

A simple measure of similarity between two embedding vectors $\mathbf{u}$ and $\mathbf{v}$ is their dot product,

$$\mathbf{u}^\top \mathbf{v} = \sum_{j=1}^d u_j v_j. \quad (15)$$

The dot product depends both on the orientation and the magnitude of the vectors. When the objective is to compare their directions independently of their magnitudes, a common alternative is the **cosine similarity**,

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2}. \quad (16)$$

Since

$$\mathbf{u}^\top \mathbf{v} = \|\mathbf{u}\|_2 \|\mathbf{v}\|_2 \cos \theta,$$

where θ is the angle between the vectors, cosine similarity reduces to

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \cos \theta.$$

For the illustrative embeddings above,

$$\mathbf{e}_{\text{cat}} = \begin{pmatrix} 0.8 \\ 0.7 \end{pmatrix}, \quad \mathbf{e}_{\text{dog}} = \begin{pmatrix} 0.9 \\ 0.6 \end{pmatrix},$$

their dot product is

$$\mathbf{e}_{\text{cat}}^\top \mathbf{e}_{\text{dog}} = (0.8)(0.9) + (0.7)(0.6) = 1.14.$$

Their norms are

$$\|\mathbf{e}_{\text{cat}}\|_2 = \sqrt{0.8^2 + 0.7^2} = \sqrt{1.13},$$

and

$$\|\mathbf{e}_{\text{dog}}\|_2 = \sqrt{0.9^2 + 0.6^2} = \sqrt{1.17}.$$

Therefore,

$$\text{sim}(\mathbf{e}_{\text{cat}}, \mathbf{e}_{\text{dog}}) = \frac{1.14}{\sqrt{1.13}\sqrt{1.17}} \approx 0.991.$$

The two vectors therefore point in very similar directions.

This geometric structure is fundamentally different from the bag-of-words representation. In a bag-of-words vocabulary space, distinct words correspond to orthogonal basis vectors. In an embedding space, different words can occupy nearby directions, allowing relationships between them to be represented numerically.

3.2 Learning the Representation

The coordinates of useful embeddings are not normally specified manually. They are **learned from data**.

The central idea is that the contexts in which words occur provide information about their relationships. Words that occur in similar linguistic contexts can acquire similar vector representations during training.

The embedding function

$$E : \mathcal{V} \rightarrow \mathbb{R}^d$$

is therefore not merely an arbitrary encoding of vocabulary terms. Its parameters are adjusted through a learning procedure so that the geometry of the resulting vector space captures statistical relationships present in the training data.

One influential example is **word2vec**, which learns word embeddings from relationships between neighbouring words in a corpus. The details of this learning procedure will be studied separately when embedding models are considered in greater depth.

The transition from bag-of-words to dense embeddings can therefore be summarised as

Bag-of-words	→	sparse, high-dimensional, independent coordinates,
Embeddings	→	dense, lower-dimensional, learned representations.

This is an important conceptual shift: rather than merely recording the occurrence of linguistic symbols, the representation itself can now encode relationships learned from data.

4 Context Matters

Dense word embeddings introduce a geometric structure in which relationships between words can be represented numerically. However, the representations considered so far have an important limitation: each vocabulary term is associated with a single fixed vector.

Consider the word *bank* in the following sentences:

She deposited the money in the bank.

They sat on the bank of the river.

The same word appears in both sentences, but its meaning is different. In the first sentence, *bank* refers to a financial institution, whereas in the second it refers to the land alongside a river.

With a static word embedding, both occurrences are mapped to the same vector:

$$E(\text{bank}) = \mathbf{e}_{\text{bank}}.$$

The representation therefore depends only on the identity of the word and not on the context in which it appears.

This reveals a more general limitation. Natural language is **context-dependent**: the meaning and function of a linguistic unit can depend on the surrounding sequence. A representation that assigns one fixed vector to each vocabulary term cannot explicitly distinguish between different uses of the same word.

A more expressive representation should therefore depend not only on the current word w_i , but also on its context.

Let a sequence of tokens be

$$S = (w_1, w_2, \dots, w_N).$$

Instead of representing w_i using only a static mapping

$$\mathbf{e}_i = E(w_i),$$

we seek a contextual representation of the form

$$\mathbf{h}_i = F(w_i, S), \tag{17}$$

where $\mathbf{h}_i$ depends on both the token w_i and the sequence in which it occurs.

For the two occurrences of *bank*, this means that

$$\mathbf{h}_{\text{bank}}^{(1)} = F(\text{bank}, \text{"she deposited the money in the bank"}),$$

and

$$\mathbf{h}_{\text{bank}}^{(2)} = F(\text{bank}, \text{"they sat on the bank of the river"}).$$

Although the token is identical,

$$w_{\text{bank}}^{(1)} = w_{\text{bank}}^{(2)},$$

the surrounding sequences are different, and therefore the resulting contextual representations need not be equal:

$$\mathbf{h}_{\text{bank}}^{(1)} \neq \mathbf{h}_{\text{bank}}^{(2)}.$$

The representation of a token has therefore changed from a fixed property of the vocabulary term to a function of the sequence in which that token appears.

4.1 From Static to Contextual Representations

This distinction can be summarised as

$$\text{Static embedding : } w_i \mapsto \mathbf{e}_i,$$

$$\text{Contextual representation : } (w_i, S) \mapsto \mathbf{h}_i.$$

The challenge is now computational: **how can information from the surrounding sequence be incorporated into the representation of each token?**

One historical approach was to process the sequence recurrently. A recurrent neural network updates an internal hidden state as it moves through the sequence, allowing information from previously processed tokens to influence later representations.

For a sequence of input representations

$$\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_N,$$

this process can be expressed abstractly as

$$\mathbf{h}_i = f(\mathbf{e}_i, \mathbf{h}_{i-1}), \tag{18}$$

where $\mathbf{h}_{i-1}$ contains information accumulated from the preceding part of the sequence.

Recurrent models therefore introduced a mechanism for constructing representations that depend on context. However, processing information sequentially also creates important limitations. Information must propagate through successive hidden states, and the sequential computation makes it difficult to process all positions in parallel.

These limitations motivated a different mechanism for incorporating context: rather than forcing information to pass sequentially through the entire sequence, a model can determine which other positions are relevant when constructing a representation.

This mechanism is known as **attention**.

5 Attention

Recurrent models provide a mechanism for incorporating context into token representations, but they process a sequence through a succession of hidden states. In an encoder-decoder architecture, this creates a particularly important limitation when the information required by the decoder must be compressed into a fixed representation of the input sequence.

Consider an input sequence

$$S = (w_1, w_2, \dots, w_N),$$

processed by an encoder into a sequence of hidden states

$$\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_N.$$

Rather than forcing the decoder to rely on a single fixed representation of the entire input, **attention** allows it to construct a context representation by selectively combining information from different encoder states.

For a decoder step t , this idea can be expressed abstractly as

$$\mathbf{c}_t = \sum_{i=1}^N \alpha_{ti} \mathbf{h}_i, \tag{19}$$

where $\mathbf{c}_t$ is the context vector and α_{ti} determines the contribution of the encoder state $\mathbf{h}_i$ at decoder step t .

The attention weights satisfy

$$\alpha_{ti} \geq 0,$$

and are typically normalised so that

$$\sum_{i=1}^N \alpha_{ti} = 1. \quad (20)$$

The context vector is therefore a weighted combination of the encoder states. Positions receiving larger attention weights contribute more strongly to the representation used at the current decoding step.

The essential difference can be expressed as

$$\text{Fixed context : } (\mathbf{h}_1, \dots, \mathbf{h}_N) \mapsto \mathbf{c},$$

$$\text{Attention : } (\mathbf{h}_1, \dots, \mathbf{h}_N) \mapsto \mathbf{c}_t.$$

With attention, the representation of the input can therefore change according to what information is relevant at a particular decoding step.

5.1 Attention Weights

The attention weights are not specified manually. They are determined from the relationship between the current decoding state and the encoder states.

Abstractly, an alignment score can be written as

$$s_{ti} = a(\mathbf{s}_{t-1}, \mathbf{h}_i), \quad (21)$$

where $\mathbf{s}_{t-1}$ represents the decoder state and a is a function that measures the relevance of encoder state $\mathbf{h}_i$ for the current decoding step.

These scores can then be converted into normalised attention weights,

$$\alpha_{ti} = \frac{\exp(s_{ti})}{\sum_{j=1}^N \exp(s_{tj})}. \quad (22)$$

Consequently,

$$\sum_{i=1}^N \alpha_{ti} = 1,$$

and the context vector becomes

$$\mathbf{c}_t = \sum_{i=1}^N \alpha_{ti} \mathbf{h}_i.$$

The mechanism therefore consists of three conceptual operations:

1. Compare the current state with the available representations.
2. Assign a relevance score to each representation.
3. Construct a weighted combination according to those scores.

This allows the model to access different parts of the input sequence directly rather than requiring all relevant information to be represented by a single fixed vector.

5.2 From Attention to Transformers

Attention was initially introduced as a mechanism used together with recurrent architectures. Recurrence, however, still requires the sequence to be processed step by step:

$$\mathbf{h}_1 \rightarrow \mathbf{h}_2 \rightarrow \dots \rightarrow \mathbf{h}_N.$$

This sequential dependency limits parallel computation because the state $\mathbf{h}_i$ generally depends on the preceding state $\mathbf{h}_{i-1}$.

This raises a natural question:

If attention provides a direct mechanism for relating different positions in a sequence, is recurrence necessary at all?

The **Transformer** architecture answers this question by removing recurrence and using attention as its central mechanism for exchanging information between positions in a sequence.

6 The Transformer

Attention provides a mechanism for directly combining information from different positions in a sequence. However, when attention is added to a recurrent architecture, the underlying recurrence remains: hidden states must still be computed sequentially.

For a recurrent model,

$$\mathbf{h}_i = f(\mathbf{e}_i, \mathbf{h}_{i-1}),$$

so the computation of $\mathbf{h}_i$ depends on the previous state $\mathbf{h}_{i-1}$. The sequence therefore contains an explicit computational dependency:

$$\mathbf{h}_1 \rightarrow \mathbf{h}_2 \rightarrow \dots \rightarrow \mathbf{h}_N.$$

Even if attention later provides direct access to these states, their construction remains sequential.

The **Transformer** introduces a different architecture. Rather than using recurrence to propagate information through a sequence, it uses attention as the principal mechanism for allowing different positions to exchange information.

Conceptually, the transition can be expressed as

Recurrent model : context through sequential state propagation,

Transformer : context through direct attention between positions.

Removing recurrence has an important computational consequence. Since the representation of position i no longer needs to wait for the recurrent state of position $i - 1$, multiple positions can be processed in parallel during training.

This ability to combine contextual modelling with parallel computation is one of the central reasons why the Transformer became the foundation of modern large language models.

6.1 Encoder-Decoder Architecture

The original Transformer follows an **encoder-decoder architecture**.

Given an input sequence

$$S = (w_1, w_2, \dots, w_N),$$

the encoder transforms the input representations into a sequence of contextual representations,

$$(\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_N) \mapsto (\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_N).$$

Unlike a recurrent encoder, the Transformer encoder allows each position to incorporate information from other positions through **self-attention**.

The term *self-attention* indicates that the elements attending to one another belong to the same sequence. Thus, when constructing the representation of a particular token, the model can use information from other tokens in that sequence.

For example, in

The animal did not cross the street because it was tired.

the representation associated with *it* can incorporate information from other positions in the sentence, including *animal*. The resulting representation is therefore contextual rather than determined exclusively by the token *it* itself.

The decoder produces the output sequence. It also uses self-attention, but when generating text it must prevent a position from accessing future output tokens. Otherwise, during training the model could use information that would not be available when generating the sequence.

For an output sequence

$$Y = (y_1, y_2, \dots, y_T),$$

the representation used to predict y_t may depend on

$$y_1, y_2, \dots, y_{t-1},$$

but not on

$$y_{t+1}, \dots, y_T.$$

This restriction is implemented through **masked self-attention**.

In an encoder-decoder Transformer, the decoder also needs access to the representations produced by the encoder. It therefore contains an additional attention mechanism that allows decoder positions to attend to the encoded input sequence.

The architecture can therefore be summarised as

Encoder : self-attention + feed-forward transformations,

Decoder : masked self-attention + encoder-decoder attention + feed-forward transformations.

6.2 Self-Attention as Contextualisation

The fundamental role of self-attention can be understood as a transformation from initial token representations to contextual token representations.

Before contextualisation, each token has an input representation

$$\mathbf{e}_i.$$

After information has been exchanged through self-attention, the representation becomes

$$\mathbf{h}_i = F(\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_N)_i. \quad (23)$$

Thus,

$$\mathbf{e}_i \longrightarrow \mathbf{h}_i,$$

where $\mathbf{e}_i$ represents the token before contextual interaction and $\mathbf{h}_i$ represents that token after incorporating information from the sequence.

This connects the Transformer directly to the representation problem introduced earlier. The objective is no longer merely to assign a numerical vector to each linguistic unit, but to construct representations whose values depend on the relationships between the elements of the sequence.

The Transformer provides the architecture within which these contextual representations can be constructed efficiently using attention.

The precise mechanism by which self-attention computes these representations will be studied later. In particular, the roles of **queries**, **keys**, and **values**, the scaled dot-product attention operation, multi-head attention, positional information, and the internal structure of Transformer blocks require a more detailed treatment.

For now, the essential architectural transition is

$$\begin{aligned} \text{static representation} &\longrightarrow \text{context-dependent representation,} \\ \text{sequential recurrence} &\longrightarrow \text{attention-based interaction.} \end{aligned}$$

7 Representation and Generative Models

The original Transformer contains both an encoder and a decoder, but many language models do not require the complete encoder-decoder architecture. Depending on the task, different parts of the Transformer can be used independently.

This leads to two important families of language models:

Encoder-only $\longrightarrow$ representation models,

Decoder-only $\longrightarrow$ generative models.

The distinction reflects two different computational objectives. A representation model primarily transforms text into contextual numerical representations, whereas a generative model predicts and produces sequences of tokens.

7.1 Encoder-Only Models

An encoder-only model receives a sequence of tokens and constructs contextual representations by allowing information to flow between positions through self-attention.

Given a token sequence

$$S = (w_1, w_2, \dots, w_N),$$

the encoder produces

$$(\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_N),$$

where each $\mathbf{h}_i$ is a contextual representation of token w_i .

Abstractly,

$$F_{\text{enc}} : (w_1, \dots, w_N) \mapsto (\mathbf{h}_1, \dots, \mathbf{h}_N). \quad (24)$$

Since the objective is to construct representations of the input rather than generate the sequence autoregressively, an encoder can generally use information from both preceding and following positions when contextualising a token.

Consider the sentence

The bank approved the loan.

When constructing the representation of *bank*, the encoder can use information from the entire sequence, including *approved* and *loan*. This contextual information helps distinguish this occurrence from the word *bank* in a sentence such as

They walked along the bank of the river.

The resulting contextual representations can subsequently be used for tasks such as classification, semantic similarity, clustering, and retrieval.

A prominent example of an encoder-only Transformer is **BERT**.

7.1.1 Masked Language Modelling

BERT was pretrained using a task known as **masked language modelling**. Some tokens in an input sequence are hidden from the model, which must predict them using the surrounding context.

For example,

The cat is [MASK] on the sofa.

can be interpreted as a prediction problem of the form

$$P(w_i \mid w_1, \dots, w_{i-1}, w_{i+1}, \dots, w_N). \quad (25)$$

The prediction can therefore depend on context appearing on both sides of the masked position.

The importance of this training procedure is not merely the prediction of missing words. Through the prediction task, the model learns internal representations that can subsequently be reused for other language tasks.

This illustrates the general principle of **transfer learning**: a model can first learn broadly useful representations from a large corpus and then be adapted to a more specific task.

7.2 Decoder-Only Models

A decoder-only model solves a different problem. Rather than constructing unrestricted representations of an existing sequence, it is designed to predict the continuation of a sequence.

Given the preceding tokens

$$w_1, w_2, \dots, w_{t-1},$$

the model estimates a probability distribution over the next token:

$$P(w_t \mid w_1, w_2, \dots, w_{t-1}). \quad (26)$$

A token can then be selected from this distribution and appended to the sequence. Repeating the process produces

$$w_1 \rightarrow w_2 \rightarrow \dots \rightarrow w_T.$$

The model is therefore **autoregressive**: each new prediction is conditioned on the tokens that precede it.

For an entire sequence,

$$S = (w_1, w_2, \dots, w_N),$$

the chain rule of probability gives

$$P(S) = P(w_1, w_2, \dots, w_N) = \prod_{i=1}^N P(w_i \mid w_1, \dots, w_{i-1}). \quad (27)$$

This factorisation provides the probabilistic foundation for autoregressive language modelling.

The architecture must respect this direction of information flow. When predicting token w_t , the model must not have access to future tokens

$$w_{t+1}, \dots, w_N.$$

Decoder-only Transformers enforce this restriction through masked, or **causal**, self-attention.

A prominent family of decoder-only Transformers is **GPT**.

7.3 Two Different Objectives

The distinction between representation and generative models can therefore be understood in terms of the information available to each position and the objective for which the model is trained:

Encoder-only : understand and represent an observed sequence,

Decoder-only : predict and generate a sequence autoregressively.

These architectures are not simply different implementations of the same operation. Their attention patterns and training objectives reflect different uses of language models.

Encoder-only models are particularly natural when the complete input is available and the objective is to construct useful representations of it. Decoder-only models are particularly natural when the objective is to generate a continuation conditioned on previously available tokens.

8 Pretraining and Fine-Tuning

Modern language models are generally not trained from scratch for every individual application. Instead, training is divided into stages that separate the acquisition of broad language capabilities from adaptation to more specific tasks or behaviours.

The two principal stages are **pretraining** and **fine-tuning**, often considered part of a broader **post-training** process.

Conceptually,

large general corpus $\longrightarrow$ pretraining $\longrightarrow$ base model $\longrightarrow$ post-training $\longrightarrow$ adapted model.

8.1 Pretraining

During pretraining, a model is exposed to a large and diverse collection of text. The objective is not initially to solve one particular downstream task, but to learn statistical regularities that are useful across many language problems.

For an autoregressive language model, this can be formulated as next-token prediction. Given a sequence

$$S = (w_1, w_2, \dots, w_N),$$

the model assigns the probability

$$P_\theta(S) = \prod_{i=1}^N P_\theta(w_i \mid w_1, \dots, w_{i-1}), \quad (28)$$

where θ denotes the model parameters.

Given a training corpus

$$\mathcal{D} = \{S^{(1)}, S^{(2)}, \dots, S^{(M)}\},$$

maximum-likelihood training seeks parameters that maximise the probability assigned to the observed sequences:

$$\hat{\theta} = \operatorname{argmax}_{\theta} \prod_{m=1}^M P_{\theta}(S^{(m)}). \quad (29)$$

Products of many probabilities are inconvenient numerically and analytically. Taking the logarithm converts the product into a sum without changing the location of the maximum because the logarithm is strictly increasing:

$$\hat{\theta} = \operatorname{argmax}_{\theta} \sum_{m=1}^M \log P_{\theta}(S^{(m)}).$$

Using the autoregressive factorisation,

$$\hat{\theta} = \operatorname{argmax}_{\theta} \sum_{m=1}^M \sum_{i=1}^{N_m} \log P_{\theta}(w_i^{(m)} \mid w_1^{(m)}, \dots, w_{i-1}^{(m)}),$$

where N_m is the number of tokens in sequence $S^{(m)}$.

Equivalently, maximising the log-likelihood is the same as minimising the **negative log-likelihood**,

$$\mathcal{L}(\theta) = - \sum_{m=1}^M \sum_{i=1}^{N_m} \log P_{\theta}(w_i^{(m)} \mid w_1^{(m)}, \dots, w_{i-1}^{(m)}). \quad (30)$$

Training therefore adjusts the model parameters so that tokens actually observed in the training corpus receive increasingly high conditional probability given their preceding context.

The resulting model is commonly referred to as a **base model** or **foundation model**. It has learned broadly useful statistical structure from the pretraining data, but it has not necessarily been adapted to a particular application or desired interaction behaviour.

8.2 Fine-Tuning

Rather than training a new model from randomly initialised parameters, a pretrained model can be adapted using a smaller and more specialised dataset.

Let

$$\hat{\theta}_{\text{pre}}$$

denote the parameters obtained during pretraining. Fine-tuning starts from these learned parameters,

$$\theta_0 = \hat{\theta}_{\text{pre}},$$

and continues optimisation using a task-specific or behaviour-specific dataset

$$\mathcal{D}_{\text{ft}}.$$

Conceptually,

$$\hat{\theta}_{\text{pre}} \longrightarrow \mathcal{D}_{\text{ft}} \longrightarrow \hat{\theta}_{\text{ft}}.$$

The objective is to retain useful representations and capabilities acquired during pretraining while adapting the model to the desired task or behaviour.

For example, a pretrained representation model can be fine-tuned for text classification, while a pretrained generative model can be further trained to follow instructions or produce outputs with particular characteristics.

This is an application of **transfer learning**: knowledge acquired from a broad source problem is reused as the starting point for a more specialised problem.

8.3 Post-Training

For modern generative language models, adaptation may involve more than conventional supervised fine-tuning. A broader term, **post-training**, can be used for the stages applied after pretraining to shape the behaviour of a base model.

At a high level,

Pretraining $\longrightarrow$ general language capabilities,
 Post-training $\longrightarrow$ task and behaviour adaptation.

Post-training may include supervised fine-tuning and additional procedures intended to make model outputs better aligned with desired behaviours.

The important distinction is therefore not simply between two datasets, but between two objectives: **pretraining develops broad capabilities from large-scale data, whereas post-training adapts those capabilities to more specific uses and behaviours.**

9 Using a Pretrained Language Model

The previous sections describe how language models represent, contextualise, and generate language. In practice, however, most applications do not begin by training a language model from scratch. Instead, a pretrained model is loaded and used directly for inference or subsequently adapted to a particular task.

For a generative language model, inference can be viewed as a sequence of transformations:

text $\longrightarrow$ tokens $\longrightarrow$ model $\longrightarrow$ token probabilities $\longrightarrow$ generated tokens $\longrightarrow$ text.

The first step is performed by a **tokenizer**, which converts the input text into a sequence of discrete token identifiers,

$$S = (w_1, w_2, \dots, w_N) \longrightarrow (t_1, t_2, \dots, t_N),$$

where each t_i is an integer identifying a token in the model's vocabulary.

The language model receives these token identifiers and internally transforms them into vector representations. Through successive Transformer layers, these representations are contextualised using the preceding sequence.

For an autoregressive model, the final representation at the current position is used to produce a set of numerical scores over the vocabulary. Let

$$\mathbf{z} = (z_1, z_2, \dots, z_{|\mathcal{V}|})$$

denote these scores, commonly called **logits**.

The logits are converted into a probability distribution using the softmax function,

$$P(w = j \mid w_{<t}) = \frac{\exp(z_j)}{\sum_{k=1}^{|V|} \exp(z_k)}. \quad (31)$$

Thus,

$$\sum_{j=1}^{|V|} P(w = j \mid w_{<t}) = 1.$$

A decoding procedure then selects the next token from this distribution. The selected token is appended to the sequence and becomes part of the context for the next prediction:

$$(w_1, \dots, w_t) \longrightarrow P(w_{t+1} \mid w_1, \dots, w_t) \longrightarrow w_{t+1}.$$

Repeating this operation generates a sequence autoregressively:

$$w_1, w_2, \dots, w_N \longrightarrow w_{N+1} \longrightarrow w_{N+2} \longrightarrow \dots.$$

Finally, the generated token identifiers are decoded back into text.

9.1 Generation Pipeline

At a high level, text generation can therefore be described by the following procedure:

Input: a text prompt S and a pretrained autoregressive language model.

Output: a generated text sequence.

1. Tokenise the input prompt.
2. Convert the tokens into the representation expected by the model.
3. Perform a forward pass through the Transformer.
4. Obtain the logits for the next token.
5. Convert the logits into a probability distribution.
6. Select the next token according to a decoding strategy.
7. Append the selected token to the current sequence.
8. Repeat the procedure until a stopping condition is reached.
9. Decode the resulting token sequence back into text.

The decoding strategy determines how a token is selected from the predicted distribution. A simple strategy is to choose the token with the largest probability,

$$w_t = \operatorname{argmax}_{w \in \mathcal{V}} P(w \mid w_{<t}), \quad (32)$$

which is known as **greedy decoding**.

Other strategies can sample from the probability distribution and control the diversity of the generated sequence. These methods will be considered later when text generation is studied in greater detail.

The important point at this stage is that generation is not performed by producing an entire response in a single operation. An autoregressive language model repeatedly predicts a distribution over the next token, selects a token, extends the context, and performs the process again.

9.2 Python Implementation

The autoregressive generation procedure can be examined directly using a pretrained causal language model. Rather than relying immediately on a high-level generation interface, the first implementation performs greedy decoding explicitly.

At each iteration, the current token sequence is passed through the model. The logits associated with the final position are extracted and converted into probabilities using the softmax function. The token with the largest probability is then selected and appended to the sequence.

Thus, the implementation follows directly the procedure

tokenise $\longrightarrow$ forward pass $\longrightarrow$ logits $\longrightarrow$ probabilities $\longrightarrow$ argmax $\longrightarrow$ next token.

This operation is repeated autoregressively until the requested number of tokens has been generated or an end-of-sequence token is produced.

The complete implementation is available in `python/text_generation.py`.

The same generation procedure can subsequently be performed using the standard `generate()` method provided by the Transformers library. Comparing the two approaches makes explicit which operations are handled internally by the higher-level interface.

Using the prompt

Machine learning is

both the explicit implementation and the standard `generate()` method produce the same sequence when greedy decoding is used. This illustrates that the higher-level generation interface encapsulates the same autoregressive procedure implemented explicitly above.

The quality of the generated text is not the objective of this example. A deliberately small pre-trained model is used so that the generation mechanism can be examined with modest computational requirements.

10 Tokenisation

A language model does not operate directly on raw text. Before a sequence can be processed by the model, it must be decomposed into smaller discrete units called **tokens**.

Given an input text S , a tokenizer transforms it into a sequence

$$T(S) = (t_1, t_2, \dots, t_N), \quad (33)$$

where T denotes the tokenisation procedure and each t_i belongs to a finite vocabulary

$$\mathcal{V} = \{v_1, v_2, \dots, v_{|\mathcal{V}|}\}.$$

Tokens do not necessarily correspond to complete words. Depending on the tokenisation scheme, they may represent words, parts of words, individual characters, or bytes.

For example, the word

apologizing

may be represented using subword tokens such as

(apolog, izing).

This decomposition is important because the vocabulary of a language model must be finite, while the number of possible words and textual expressions is effectively unbounded.

10.1 Token IDs

Although tokenisation decomposes text into discrete units, the language model does not receive the textual tokens themselves. Each token in the vocabulary is associated with a unique integer called a **token ID**.

An indexing function can therefore be defined as

$$I : \mathcal{V} \longrightarrow \{0, 1, \dots, |\mathcal{V}| - 1\},$$

such that

$$I(t_i) = i_i.$$

The complete transformation performed before the text enters the language model can then be represented as

$$S \longrightarrow (t_1, t_2, \dots, t_N) \longrightarrow (i_1, i_2, \dots, i_N).$$

The resulting sequence

$$\mathbf{i} = (i_1, i_2, \dots, i_N)$$

is the discrete numerical input presented to the language model.

A token ID should not be interpreted as a numerical measurement. The magnitude of the integer carries no semantic information: token $i = 500$ is not inherently more similar to token $i = 501$ than to token $i = 10$. The ID simply identifies an entry in the tokenizer vocabulary.

The token IDs will subsequently be used by the language model to retrieve vectors from its embedding matrix. Thus, the complete input pipeline can be summarized as

$$\text{text} \longrightarrow \text{tokens} \longrightarrow \text{token IDs} \longrightarrow \text{embeddings}.$$

10.2 Encoding and Decoding

The tokenizer provides the interface between human-readable text and the discrete representation used by the language model.

Encoding transforms an input string into token IDs,

$$\text{encode}(S) = (i_1, i_2, \dots, i_N),$$

whereas **decoding** performs the reverse operation,

$$\text{decode}(i_1, i_2, \dots, i_N) = \hat{S}.$$

Conceptually,

$$S \xrightarrow{\text{encode}} (i_1, \dots, i_N) \xrightarrow{\text{decode}} \hat{S}.$$

The same tokenizer is therefore involved at both ends of a generative language model. It encodes the input text before model processing and decodes the generated token IDs back into human-readable text.

10.3 Tokenisation Granularity

A fundamental design choice in tokenisation is the **granularity** at which text is decomposed. A token may correspond to an entire word, a part of a word, an individual character, or a byte.

For an input string S , different tokenisation schemes therefore produce different sequences,

$$T(S) = (t_1, t_2, \dots, t_N),$$

with potentially different sequence lengths N and vocabulary sizes $|\mathcal{V}|$.

The choice of granularity introduces an important trade-off. Larger linguistic units generally produce shorter sequences but require larger vocabularies, whereas smaller units reduce the vocabulary requirements but produce longer sequences.

Conceptually,

$$\text{larger tokens} \longrightarrow \text{larger vocabulary, shorter sequences,}$$

whereas

smaller tokens $\rightarrow$ smaller vocabulary, longer sequences.

10.3.1 Word Tokens

Word-level tokenisation treats complete words as the fundamental units of the sequence. For example,

The cat is sleeping

may be represented as

(The, cat, is, sleeping).

This representation is linguistically intuitive and produces relatively short sequences. However, it requires the vocabulary to contain the words that the tokenizer must represent.

This creates an **out-of-vocabulary problem**. A word that was not included when the vocabulary was constructed cannot be represented directly and may need to be mapped to a special unknown token.

Word-level vocabularies may also contain many closely related forms as separate entries. For example,

{apology, apologize, apologetic, apologist}

require distinct vocabulary entries even though the words share substantial lexical structure.

10.3.2 Subword Tokens

Subword tokenisation provides an intermediate representation between complete words and individual characters. Frequently occurring words or word fragments can be represented directly, while less common or previously unseen words can be decomposed into smaller units.

For example,

apologizing $\rightarrow$ (apolog, izing).

The same subword components can be reused across multiple words. This allows the tokenizer to represent a much larger set of possible words without explicitly storing every complete word in its vocabulary.

More generally, a word w may be represented as

$$w = t_1 \oplus t_2 \oplus \cdots \oplus t_K,$$

where $\oplus$ denotes the reconstruction of the original string from its subword tokens.

This makes subword tokenisation particularly useful for language models: common linguistic units can remain compact, while uncommon words can still be represented through smaller components.

Several widely used tokenisation algorithms construct subword vocabularies, including **Byte Pair Encoding (BPE)** and **WordPiece**. The precise algorithms used to construct these vocabularies differ, but they share the objective of finding a useful set of reusable units for representing the training corpus.

10.3.3 Character Tokens

Character-level tokenisation decomposes text into individual characters. For example,

$$\text{play} \rightarrow (\text{p}, \text{l}, \text{a}, \text{y}).$$

Because the number of possible characters is much smaller than the number of possible words, character-level tokenisation can operate with a relatively small vocabulary and can naturally represent previously unseen words.

The cost is an increase in sequence length. A word represented by one subword token may require several character tokens.

If a text contains C characters and each character is represented individually, the resulting sequence length is approximately

$$N \approx C.$$

Longer token sequences increase the amount of sequence that the model must process and consume more of its finite context window. The model must also learn relationships spanning multiple character positions in order to represent linguistic units that a word- or subword-level tokenizer may encode directly.

10.3.4 Byte Tokens

Tokenisation can be performed at an even lower level by representing text through the bytes used to encode its characters.

For a byte-level representation,

$$S \longrightarrow (b_1, b_2, \dots, b_N),$$

where each b_i represents a byte of the encoded input.

Byte-level representations provide a small fundamental alphabet and can represent arbitrary text supported by the underlying character encoding, including multilingual characters and other symbols.

The trade-off is again sequence length: characters represented by multiple bytes require multiple elements in the resulting sequence.

Byte-level modelling should be distinguished from subword tokenizers that use bytes only as a fallback mechanism. A tokenizer may primarily construct subword tokens while retaining byte-level units for characters that cannot otherwise be represented. This does not make the model purely byte-level.

10.4 The Granularity Trade-off

The four approaches can be viewed along a spectrum,

$$\text{word} \longrightarrow \text{subword} \longrightarrow \text{character} \longrightarrow \text{byte},$$

where moving toward smaller fundamental units generally increases the ability to represent arbitrary input but also tends to increase sequence length.

Subword tokenisation provides a practical compromise. It can preserve frequent words or meaningful fragments as individual tokens while decomposing uncommon words into smaller reusable units.

For Transformer-based language models, this trade-off is particularly important because the model operates over a finite number of tokens. Tokenisation therefore affects not only how text is represented, but also how much text can fit within the model's context window.

10.5 Tokenizer Design

Tokenisation is not determined solely by whether a model uses words, subwords, characters, or bytes. Even among tokenizers based on similar schemes, the resulting token sequences can differ substantially.

Three major design factors determine how a tokenizer represents text:

1. the **tokenisation method** used to construct the vocabulary;
2. the **tokenizer parameters**, including vocabulary size and special tokens;
3. the **domain of the data** used to train the tokenizer.

Consequently, tokenisation should be regarded as a learned and designed component of the language-model pipeline rather than as a universal preprocessing rule.

10.5.1 Tokenisation Method

A tokenisation method defines how the vocabulary is constructed from a training corpus and, consequently, how new text is decomposed into tokens.

Two important subword methods are **Byte Pair Encoding (BPE)** and **WordPiece**. SentencePiece provides another widely used tokenization framework and supports methods such as BPE and the unigram language model.

Although these approaches differ in how they construct and apply their vocabularies, they share the general objective of finding a finite set of reusable units that can efficiently represent the training corpus.

Thus, the tokenisation method determines the procedure by which

$$\mathcal{D} \longrightarrow \mathcal{V},$$

where $\mathcal{D}$ denotes the tokenizer training corpus and $\mathcal{V}$ the resulting vocabulary.

Once the vocabulary has been constructed, the trained tokenizer defines a mapping

$$T_{\mathcal{V}} : \mathcal{S} \longrightarrow \mathcal{V}^*,$$

which decomposes new input strings into sequences of vocabulary elements.

10.5.2 Vocabulary Size

The vocabulary size

$$|\mathcal{V}|$$

is another important design parameter.

A larger vocabulary can represent more frequently occurring strings as individual tokens, potentially reducing the number of tokens required to encode a given text. However, it also requires the model to maintain representations for a larger number of distinct tokens.

Conversely, a smaller vocabulary relies more heavily on decomposing text into reusable sub-word units.

Vocabulary size therefore interacts directly with tokenisation granularity and sequence length. It is not an independent property of the language model.

10.5.3 Special Tokens

Not every token represents ordinary textual content. A tokenizer may reserve **special tokens** that communicate structural or task-specific information to the model.

Common examples include tokens representing

- the beginning or end of a sequence;
- padding;
- unknown input;
- masked positions;
- classification;
- separators between different pieces of text.

For example, BERT-style tokenizers may introduce tokens such as

[CLS], [SEP], [MASK], [PAD],

whereas conversational language models may define tokens representing different roles in a dialogue, such as

<|system|>, <|user|>, <|assistant|>.

Special tokens therefore extend the vocabulary beyond lexical units. They allow information about sequence structure, model tasks, or interaction protocols to be represented within the same discrete token sequence processed by the model.

10.5.4 Capitalisation and Whitespace

Tokenizers may also differ in how they preserve information such as capitalisation and whitespace.

For example, an **uncased** tokenizer may transform uppercase characters into lowercase before tokenisation. This reduces the number of distinct forms that must be represented, but removes information encoded by capitalisation.

Whitespace can likewise contain relevant structure. In natural-language text, multiple spaces or line breaks may carry relatively little information in some applications, whereas in source code indentation can be syntactically significant.

Tokenisation choices therefore determine which properties of the original input remain explicitly visible to the language model.

10.5.5 Training-Data Domain

The vocabulary produced by a tokenizer depends on the corpus on which the tokenizer itself is trained.

Let

$$\mathcal{D}_{\text{tok}}$$

denote the tokenizer training corpus. Then the learned vocabulary can be viewed abstractly as

$$\mathcal{V} = A(\mathcal{D}_{\text{tok}}; \lambda),$$

where A denotes the tokenisation algorithm and λ represents its design parameters.

Therefore, even when two tokenizers use the same algorithm and parameters,

$$\mathcal{D}_{\text{tok}}^{(1)} \neq \mathcal{D}_{\text{tok}}^{(2)}$$

may lead to

$$\mathcal{V}^{(1)} \neq \mathcal{V}^{(2)}.$$

A tokenizer trained primarily on natural-language text may therefore behave differently from one trained on source code or multilingual data.

For example, a code-oriented tokenizer may reserve efficient representations for repeated whitespace, programming-language keywords, or other structures that occur frequently in source code. This can reduce the number of tokens required to represent those structures and make them more directly available to the language model.

The tokenizer is therefore adapted not only to a language model architecture but also to the statistical structure of the data that the model is expected to process.

10.6 Python Implementation

The tokenisation pipeline can be inspected directly using a pretrained tokenizer. The following implementation separates the main operations involved in transforming text into the discrete representation expected by a language model.

Given an input string, the tokenizer first encodes the text as a sequence of token IDs. These IDs can then be mapped back to their corresponding tokens or decoded collectively to reconstruct the text.

The complete implementation is available in `python/tokenization.py`.

For an input text S , the implementation exposes the sequence

$$S \longrightarrow (t_1, t_2, \dots, t_N) \longrightarrow (i_1, i_2, \dots, i_N),$$

and subsequently applies

$$(i_1, i_2, \dots, i_N) \xrightarrow{\text{decode}} \hat{S}.$$

The example also reports the tokenizer vocabulary size and its special tokens. This makes it possible to inspect both the representation produced for a particular input and some of the design properties of the tokenizer itself.

11 Token Embeddings

Token IDs provide a discrete numerical representation of the tokens in a sequence, but the integer values themselves contain no semantic information. A language model therefore transforms each token ID into a continuous vector representation called a **token embedding**.

If the tokenizer vocabulary contains $|\mathcal{V}|$ tokens and each token is represented by a vector of dimension d , the language model contains an embedding matrix

$$E \in \mathbb{R}^{|\mathcal{V}| \times d}. \quad (34)$$

Each row of E corresponds to one token in the tokenizer vocabulary. For a token with ID i , its embedding is obtained by selecting the corresponding row,

$$\mathbf{e}_i = E_{i,:} \in \mathbb{R}^d. \quad (35)$$

The token ID therefore acts as an **index** into the embedding matrix rather than as a numerical feature supplied directly to the model.

For a sequence of token IDs

$$\mathbf{i} = (i_1, i_2, \dots, i_N),$$

the embedding lookup produces a sequence of vectors

$$(\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_N),$$

which can be represented as a matrix

$$X = \begin{bmatrix} \mathbf{e}_1^\top \\ \mathbf{e}_2^\top \\ \vdots \\ \mathbf{e}_N^\top \end{bmatrix} \in \mathbb{R}^{N \times d}. \quad (36)$$

The input representation of a language model can therefore be extended from the tokenisation pipeline as

$$\text{text} \longrightarrow \text{tokens} \longrightarrow \text{token IDs} \longrightarrow \text{token embeddings}.$$

11.1 The Embedding Matrix

The embedding matrix is part of the language model itself. Consequently, a pretrained language model is tied to the vocabulary of its associated tokenizer: the token IDs produced by the tokenizer determine which rows of the model's embedding matrix are retrieved.

Before training, the embedding vectors are initialized as model parameters. During training, their values are adjusted together with the other model parameters so that they become useful numerical representations for the task learned by the model.

The transformation from a token ID to its embedding can also be expressed using a one-hot representation. Let

$$\mathbf{o}_i \in \{0, 1\}^{|\mathcal{V}|}$$

be a one-hot vector whose i -th component is equal to one. Then

$$\mathbf{e}_i^\top = \mathbf{o}_i^\top E. \quad (37)$$

Because $\mathbf{o}_i$ contains only one nonzero element, this matrix multiplication simply selects the i -th row of E . In practice, embedding layers perform this operation as an efficient lookup rather than explicitly constructing the one-hot vector.

This explains why token IDs themselves do not need to encode semantic relationships. Those relationships are represented in the geometry of the learned embedding vectors rather than in the numerical values assigned to their IDs.

11.2 Contextualised Token Embeddings

The embedding matrix associates each token ID with a fixed input vector. Consequently, every occurrence of the same token ID initially retrieves the same embedding,

$$i \longrightarrow \mathbf{e}_i.$$

This representation is **static**: the lookup depends only on the token ID and not on the surrounding tokens.

However, the meaning and function of a token often depend on its context. Consider the word *bank* in the sentences

The bank approved the loan

and

They sat on the bank of the river.

A static embedding assigns the same initial vector to *bank* in both sequences. A language model processes that initial representation together with the surrounding sequence and produces a **contextualised embedding** whose value depends on the context in which the token occurs.

For an input sequence of token embeddings

$$X = (\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_N),$$

a language model can be represented abstractly as a function

$$F : \mathbb{R}^{N \times d} \longrightarrow \mathbb{R}^{N \times h},$$

such that

$$H = F(X) = (\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_N), \quad (38)$$

where

$$\mathbf{h}_i \in \mathbb{R}^h$$

is the contextualised representation of the token at position i .

Unlike the initial embedding,

$$\mathbf{e}_i = E_{i,:},$$

the contextualised representation can be expressed more generally as

$$\mathbf{h}_i = F_i(\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_N), \quad (39)$$

emphasising that the representation of a token may depend on the other tokens in the sequence.

Therefore, two occurrences of the same token may begin with the same input embedding but produce different contextualised representations,

$$\mathbf{e}_i^{(1)} = \mathbf{e}_i^{(2)},$$

while

$$\mathbf{h}_i^{(1)} \neq \mathbf{h}_i^{(2)},$$

when the surrounding contexts differ.

The distinction can be summarised as

token ID $\longrightarrow$ static token embedding $\longrightarrow$ contextualised token representation.

The mechanism by which Transformer-based language models construct these contextualised representations will be developed separately when studying the Transformer architecture.

11.2.1 Output Representation

For a batch containing B sequences of N tokens, a language model typically produces a three-dimensional tensor of contextualised representations,

$$H \in \mathbb{R}^{B \times N \times h},$$

where

- B is the batch size;
- N is the sequence length;
- h is the dimensionality of each contextualised representation.

For a single sequence,

$$B = 1,$$

so an output shape such as

$$1 \times 4 \times 384$$

represents one sequence containing four tokens, with each token represented by a vector in

$$\mathbb{R}^{384}.$$

The sequence dimension includes any special tokens inserted by the tokenizer. Thus, a two-word input may contain more than two token positions after tokenisation.

11.3 Python Implementation

The distinction between token IDs, static token embeddings, and contextualised token representations can be inspected directly in a pretrained language model.

For an input sequence, the implementation first obtains the token IDs produced by the tokenizer,

$$(i_1, i_2, \dots, i_N).$$

The model's embedding layer then maps these IDs to their initial vector representations,

$$(i_1, i_2, \dots, i_N) \longrightarrow (\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_N).$$

Finally, the complete language model processes the sequence of initial embeddings and produces contextualised representations,

$$(\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_N) \longrightarrow (\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_N).$$

The implementation also compares repeated occurrences of the same token. Their initial embeddings are identical because they correspond to the same row of the embedding matrix, whereas their contextualised representations may differ because the surrounding contexts are different.

The complete implementation is available in `python/token_embeddings.py`.

12 Text Embeddings

Token embeddings represent the individual tokens in a sequence. Many applications, however, require a numerical representation of the text as a whole rather than a separate vector for each token.

A **text embedding** represents an entire piece of text with a single vector. Given a text sequence S , a text embedding model defines a mapping

$$G : \mathcal{S} \longrightarrow \mathbb{R}^d,$$

such that

$$\mathbf{z} = G(S) \in \mathbb{R}^d, \tag{40}$$

where $\mathbf{z}$ is a fixed-dimensional vector representing the input text.

The input may contain a variable number of tokens,

$$S \longrightarrow (t_1, t_2, \dots, t_N),$$

but the resulting text embedding has a fixed dimension determined by the embedding model,

$$(t_1, t_2, \dots, t_N) \longrightarrow \mathbf{z} \in \mathbb{R}^d.$$

Thus, token embeddings and text embeddings represent different levels of the input:

$$\begin{aligned} \text{token representation:} \quad & S \longrightarrow (\mathbf{h}_1, \dots, \mathbf{h}_N), \\ \text{text representation:} \quad & S \longrightarrow \mathbf{z}. \end{aligned}$$

The objective of a text embedding model is not merely to compress a sequence into a fixed-size vector, but to construct a representation whose geometry captures useful properties of the text. Texts with related meanings can therefore occupy nearby regions of the embedding space, making these representations useful for tasks such as semantic search, categorisation, and retrieval.

12.1 Pooling Token Representations

One way to obtain a single text embedding from a sequence of token representations is to aggregate them using a **pooling operation**.

Given contextualised token representations

$$H = (\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_N),$$

a simple approach is mean pooling,

$$\mathbf{z} = \frac{1}{N} \sum_{i=1}^N \mathbf{h}_i. \quad (41)$$

This operation transforms a variable-length sequence

$$H \in \mathbb{R}^{N \times h}$$

into a single fixed-dimensional representation

$$\mathbf{z} \in \mathbb{R}^h.$$

Mean pooling illustrates the basic relationship between token-level and text-level representations, but pooling alone does not guarantee a high-quality text embedding space. Models intended specifically for text embeddings are trained so that their resulting vectors are useful representations of semantic relationships between texts.

12.2 Python Implementation

Text embeddings can be generated using a pretrained embedding model. Unlike the token-level representations examined previously, each input text is mapped directly to a single fixed-dimensional vector,

$$S \longrightarrow \mathbf{z} \in \mathbb{R}^d.$$

The geometry of the resulting embedding space can be examined by comparing vectors using **cosine similarity**. Given two text embeddings $\mathbf{z}_1$ and $\mathbf{z}_2$, their cosine similarity is

$$\text{sim}(\mathbf{z}_1, \mathbf{z}_2) = \frac{\mathbf{z}_1^\top \mathbf{z}_2}{\|\mathbf{z}_1\|_2 \|\mathbf{z}_2\|_2}. \quad (42)$$

The measure depends on the angle between the two vectors rather than directly on their magnitudes. Larger cosine similarity indicates greater alignment between their directions in the embedding space.

The implementation generates embeddings for several sentences and compares their cosine similarities. This provides a simple empirical illustration of how semantically related texts can occupy nearby regions of a learned embedding space.

The complete implementation is available in `python/text_embeddings.py`.

13 Word2vec

Word2vec is a method for learning vector representations of words from their occurrence patterns in a corpus. Rather than defining semantic relationships explicitly, the method learns embeddings from the contexts in which words appear.

The central idea is that words occurring in similar linguistic contexts should acquire useful geometric relationships in the learned embedding space.

13.1 Skip-Gram

The **skip-gram** approach constructs training examples by moving a sliding window through a sequence of words.

Consider a token sequence

$$(w_1, w_2, \dots, w_N)$$

and let c denote the context-window size. For a centre word w_i , its context is

$$C_i = \{w_j : 0 < |j - i| \leq c\}. \quad (43)$$

Each centre-context pair

$$(w_i, w_j), \quad w_j \in C_i,$$

forms a positive training example.

For example, consider the sequence

$$(\text{the}, \text{cat}, \text{sits}, \text{on}, \text{the}, \text{mat})$$

with

$$c = 2.$$

If the centre word is

$$w_i = \text{sits},$$

the context contains

$$C_i = \{\text{the, cat, on, the}\}.$$

The corresponding positive centre-context pairs are therefore

$$(\text{sits, the}), \quad (\text{sits, cat}), \quad (\text{sits, on}), \quad (\text{sits, the}).$$

By moving the window through a large corpus, a large collection of such training pairs can be generated.

13.2 Positive and Negative Examples

Positive examples alone are insufficient for learning whether two words tend to occur in the same context. A model trained only with positive examples could minimise its error by predicting that every pair is related.

Word2vec therefore augments the positive centre-context pairs with **negative samples**. These are pairs formed by combining a centre word with words sampled from the vocabulary that do not correspond to the observed context.

A training example can consequently be represented as

$$(w_i, w_j, y),$$

where

$$y = \begin{cases} 1, & \text{if } (w_i, w_j) \text{ is an observed centre-context pair,} \\ 0, & \text{if } (w_i, w_j) \text{ is a negative sample.} \end{cases} \quad (44)$$

The learning problem can therefore be interpreted as a binary classification problem: given two words, estimate whether they form an observed contextual pair.

13.3 Learning Word Embeddings

Let

$$\mathcal{V} = \{v_1, \dots, v_{|\mathcal{V}|}\}$$

be the vocabulary and let d be the embedding dimension.

Word2vec associates words with trainable vectors. For a centre word w and a context word c , denote their vectors by

$$\mathbf{u}_w \in \mathbb{R}^d$$

and

$$\mathbf{v}_c \in \mathbb{R}^d,$$

respectively.

Their compatibility can be measured using the dot product

$$s(w, c) = \mathbf{u}_w^\top \mathbf{v}_c. \quad (45)$$

A large positive value indicates that the two vectors are strongly aligned, whereas a smaller or negative value indicates weaker compatibility.

To transform this score into a value between zero and one, the logistic sigmoid function is applied,

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

The model therefore estimates

$$P(y = 1 \mid w, c) = \sigma(\mathbf{u}_w^\top \mathbf{v}_c). \quad (46)$$

For a labelled pair (w, c, y) , the binary cross-entropy loss is

$$\ell(w, c, y) = -[y \log \sigma(\mathbf{u}_w^\top \mathbf{v}_c) + (1 - y) \log (1 - \sigma(\mathbf{u}_w^\top \mathbf{v}_c))]. \quad (47)$$

Minimising this loss encourages observed centre-context pairs to receive larger compatibility scores and negative pairs to receive smaller scores.

The embedding vectors are initially random parameters. As the model processes many positive and negative examples, gradient-based optimisation modifies these vectors so that their geometry reflects statistical relationships present in the corpus.

13.4 Negative Sampling

Training a binary classifier on every possible centre-context pair would be computationally expensive because the vocabulary may contain a very large number of words. **Negative sampling** replaces this problem with a smaller binary classification task constructed for each observed centre-context pair.

Consider a positive pair

$$(w, c),$$

obtained from the skip-gram sliding window. The pair is assigned the label

$$y = 1.$$

For the same centre word w , suppose that K negative context words

$$n_1, n_2, \dots, n_K$$

are sampled from a noise distribution over the vocabulary. These produce the negative pairs

$$(w, n_1), (w, n_2), \dots, (w, n_K),$$

each with label

$$y = 0.$$

For the positive pair, the model seeks to maximise

$$P(y = 1 \mid w, c) = \sigma(\mathbf{u}_w^\top \mathbf{v}_c).$$

For a negative pair (w, n_k) , the desired probability is instead

$$P(y = 0 \mid w, n_k) = 1 - \sigma(\mathbf{u}_w^\top \mathbf{v}_{n_k}).$$

Using the identity

$$1 - \sigma(x) = \sigma(-x),$$

this probability can be written as

$$P(y = 0 \mid w, n_k) = \sigma \left(-\mathbf{u}_w^\top \mathbf{v}_{n_k} \right).$$

Assuming the sampled examples are treated independently, the objective associated with one positive pair and its K negative samples is

$$J(w, c) = \sigma \left(\mathbf{u}_w^\top \mathbf{v}_c \right) \prod_{k=1}^K \sigma \left(-\mathbf{u}_w^\top \mathbf{v}_{n_k} \right). \quad (48)$$

Maximising this product is equivalent to maximising its logarithm,

$$\log J(w, c) = \log \sigma \left(\mathbf{u}_w^\top \mathbf{v}_c \right) + \sum_{k=1}^K \log \sigma \left(-\mathbf{u}_w^\top \mathbf{v}_{n_k} \right). \quad (49)$$

Equivalently, the problem can be formulated as minimising the negative log-likelihood,

$$\mathcal{L}(w, c) = -\log \sigma \left(\mathbf{u}_w^\top \mathbf{v}_c \right) - \sum_{k=1}^K \log \sigma \left(-\mathbf{u}_w^\top \mathbf{v}_{n_k} \right). \quad (50)$$

The first term encourages the centre word and its observed context word to obtain a large dot product,

$$\mathbf{u}_w^\top \mathbf{v}_c \longrightarrow +\infty,$$

whereas the negative terms encourage randomly sampled context words to obtain smaller dot products,

$$\mathbf{u}_w^\top \mathbf{v}_{n_k} \longrightarrow -\infty.$$

The resulting embedding space is therefore learned indirectly through the prediction task. Words are not assigned semantic coordinates explicitly; their vectors are adjusted according to the contextual relationships observed throughout the corpus.

13.5 Algorithm

Skip-Gram with Negative Sampling learns word embeddings by repeatedly distinguishing observed centre-context pairs from randomly generated negative pairs.

Input: A tokenised corpus, vocabulary $\mathcal{V}$, embedding dimension d , context-window size c , number of negative samples K , learning rate η , and number of training epochs.

Output: Learned word embeddings.

1. Construct the vocabulary $\mathcal{V}$ from the corpus and assign an integer ID to each word.
2. Initialise two embedding matrices,

$$U, V \in \mathbb{R}^{|\mathcal{V}| \times d},$$

where U contains centre-word embeddings and V contains context-word embeddings.

3. Generate positive centre-context pairs using a sliding window. For each centre word w_i , construct

$$(w_i, w_j), \quad 0 < |j - i| \leq c.$$

4. For each positive pair (w, c) :
 - a. Retrieve the centre and context vectors

$$\mathbf{u}_w = U_{w,:}, \quad \mathbf{v}_c = V_{c,:}.$$

- b. Compute the positive score

$$s^+ = \mathbf{u}_w^\top \mathbf{v}_c.$$

- c. Sample K negative context words

$$n_1, n_2, \dots, n_K$$

from the vocabulary.

- d. For each negative word n_k , compute

$$s_k^- = \mathbf{u}_w^\top \mathbf{v}_{n_k}.$$

- e. Evaluate the negative-sampling loss

$$\mathcal{L} = -\log \sigma(s^+) - \sum_{k=1}^K \log \sigma(-s_k^-).$$

- f. Compute the gradients of $\mathcal{L}$ with respect to the centre, positive-context, and negative-context embeddings.
g. Update the corresponding rows of U and V using gradient descent,

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}.$$

5. Repeat the training procedure over the generated examples for the specified number of epochs.
6. Return the learned word embeddings.

13.6 Gradient Derivation

The Skip-Gram with Negative Sampling loss for one positive centre-context pair (w, c) and K negative samples is

$$\mathcal{L} = -\log \sigma(\mathbf{u}_w^\top \mathbf{v}_c) - \sum_{k=1}^K \log \sigma(-\mathbf{u}_w^\top \mathbf{v}_{n_k}).$$

To implement gradient descent directly, the gradients of this loss with respect to the embedding vectors must be determined.

For the sigmoid function,

$$\sigma(x) = \frac{1}{1 + e^{-x}},$$

the derivative is

$$\sigma'(x) = \sigma(x) [1 - \sigma(x)].$$

Therefore,

$$\frac{d}{dx} [-\log \sigma(x)] = \sigma(x) - 1, \tag{51}$$

and

$$\frac{d}{dx} [-\log \sigma(-x)] = \sigma(x). \quad (52)$$

Define the positive score as

$$s^+ = \mathbf{u}_w^\top \mathbf{v}_c$$

and the negative scores as

$$s_k^- = \mathbf{u}_w^\top \mathbf{v}_{n_k}.$$

The loss can then be written as

$$\mathcal{L} = -\log \sigma(s^+) - \sum_{k=1}^K \log \sigma(-s_k^-).$$

13.6.1 Gradient with Respect to the Centre Embedding

Using

$$\nabla_{\mathbf{u}_w} s^+ = \mathbf{v}_c$$

and

$$\nabla_{\mathbf{u}_w} s_k^- = \mathbf{v}_{n_k},$$

the gradient with respect to the centre embedding is

$$\nabla_{\mathbf{u}_w} \mathcal{L} = [\sigma(s^+) - 1] \mathbf{v}_c + \sum_{k=1}^K \sigma(s_k^-) \mathbf{v}_{n_k}. \quad (53)$$

The first term adjusts the centre embedding according to the observed context word, while the remaining terms account for the negative samples.

13.6.2 Gradient with Respect to the Context Embeddings

For the positive context embedding,

$$\nabla_{\mathbf{v}_c} \mathcal{L} = [\sigma(s^+) - 1] \mathbf{u}_w. \quad (54)$$

For each negative context embedding,

$$\nabla_{\mathbf{v}_{n_k}} \mathcal{L} = \sigma(s_k^-) \mathbf{u}_w. \quad (55)$$

The corresponding gradient-descent updates are therefore

$$\mathbf{u}_w \leftarrow \mathbf{u}_w - \eta \nabla_{\mathbf{u}_w} \mathcal{L},$$

$$\mathbf{v}_c \leftarrow \mathbf{v}_c - \eta \nabla_{\mathbf{v}_c} \mathcal{L},$$

and

$$\mathbf{v}_{n_k} \leftarrow \mathbf{v}_{n_k} - \eta \nabla_{\mathbf{v}_{n_k}} \mathcal{L}, \quad k = 1, \dots, K.$$

Repeated application of these updates across the centre-context pairs generated from the corpus gradually transforms the initially random vectors into learned word representations.

13.7 Python Implementation

Skip-Gram with Negative Sampling can be implemented directly from the objective function and gradients derived above. The implementation uses NumPy for vector operations but does not rely on a machine-learning framework or automatic differentiation.

Starting from a tokenised corpus, the implementation constructs the vocabulary and generates positive centre-context pairs using a sliding window. For every positive pair, negative context words are sampled from the vocabulary.

Two embedding matrices are initialised,

$$U, V \in \mathbb{R}^{|\mathcal{V}| \times d},$$

and trained by applying the gradients derived in Equation 53, Equation 54, and Equation 55.

After training, the learned centre embeddings in U can be compared using cosine similarity,

$$\text{sim}(\mathbf{u}_i, \mathbf{u}_j) = \frac{\mathbf{u}_i^\top \mathbf{u}_j}{\|\mathbf{u}_i\|_2 \|\mathbf{u}_j\|_2}.$$

The implementation is deliberately small and uses a simple negative-sampling procedure so that the relationship between the mathematical formulation and the numerical algorithm remains explicit.

The complete implementation is available in `python/word2vec.py`.

14 Embeddings Beyond Language

The embedding principle is not restricted to words or text. More generally, embeddings can be learned for discrete objects whenever meaningful relationships between those objects can be inferred from their patterns of occurrence.

Word2vec provides a useful example of this generalisation. In its original language setting, a sentence can be represented as a sequence of words,

$$(w_1, w_2, \dots, w_N),$$

and the algorithm learns word embeddings from the contexts in which those words occur.

The same structure can be applied to recommendation problems. Consider a collection of user-generated sequences

$$(s_1, s_2, \dots, s_N),$$

where each s_i represents an item rather than a word. For example, if the sequences are music playlists, each s_i may represent a song.

The analogy is then

$$\begin{aligned} \text{word} &\longleftrightarrow \text{item}, \\ \text{sentence} &\longleftrightarrow \text{item sequence}, \\ \text{linguistic context} &\longleftrightarrow \text{co-occurrence context}. \end{aligned}$$

Skip-gram can consequently generate centre-context pairs from item sequences in exactly the same structural manner as it generates word pairs from sentences.

After training, each item is represented by a vector

$$\mathbf{e}_i \in \mathbb{R}^d.$$

Items that tend to occur in similar contexts can acquire nearby representations in the embedding space. Candidate recommendations can then be obtained by comparing item embeddings using a similarity measure such as cosine similarity,

$$\text{sim}(\mathbf{e}_i, \mathbf{e}_j) = \frac{\mathbf{e}_i^\top \mathbf{e}_j}{\|\mathbf{e}_i\|_2 \|\mathbf{e}_j\|_2}.$$

The important abstraction is therefore not specifically

$$\text{word2vec} = \text{word embeddings},$$

but rather the more general pattern

sequences of discrete objects $\longrightarrow$ co-occurrence relationships $\longrightarrow$ learned vector representations.

This illustrates why embedding methods extend naturally beyond language and can be useful in domains such as recommendation systems.

15 Autoregressive Text Generation

A generative language model produces text sequentially, one token at a time. Given an initial sequence of tokens

$$S_N = (t_1, t_2, \dots, t_N),$$

the model processes the complete available context and produces a distribution over the vocabulary for the next token,

$$P(t_{N+1} \mid t_1, t_2, \dots, t_N). \tag{56}$$

A decoding strategy selects a token from this distribution,

$$t_{N+1} \sim P(t \mid t_1, t_2, \dots, t_N),$$

where the selection may be deterministic or stochastic depending on the decoding procedure.

The selected token is appended to the existing sequence,

$$S_{N+1} = (t_1, t_2, \dots, t_N, t_{N+1}),$$

and the updated sequence is used to predict the following token,

$$P(t_{N+2} \mid t_1, t_2, \dots, t_N, t_{N+1}).$$

The generation process therefore follows the iterative structure

$$S_N \longrightarrow P(t_{N+1} \mid S_N) \longrightarrow t_{N+1} \longrightarrow S_{N+1} \longrightarrow P(t_{N+2} \mid S_{N+1}) \longrightarrow \dots$$

This dependence on previously generated outputs makes the model **autoregressive**.

For a generated continuation

$$(t_{N+1}, t_{N+2}, \dots, t_{N+M}),$$

the conditional probability of the complete continuation can be factorised using the chain rule of probability,

$$P(t_{N+1}, \dots, t_{N+M} \mid t_1, \dots, t_N) = \prod_{m=1}^M P(t_{N+m} \mid t_1, \dots, t_{N+m-1}). \quad (57)$$

Each factor corresponds to one next-token prediction problem. Consequently, text generation can be understood as the repeated application of the same language model to an increasingly long sequence.

15.1 The Forward Pass

A **forward pass** is the sequence of computations performed by the language model to transform an input token sequence into predictions for the next token.

Given an input sequence

$$S_N = (t_1, t_2, \dots, t_N),$$

the tokenizer maps the tokens to their corresponding token IDs,

$$(i_1, i_2, \dots, i_N).$$

The embedding matrix

$$E \in \mathbb{R}^{|\mathcal{V}| \times d}$$

then maps each token ID to an initial vector representation. The resulting input matrix is

$$X^{(0)} = \begin{bmatrix} \mathbf{e}_1^\top \\ \mathbf{e}_2^\top \\ \vdots \\ \mathbf{e}_N^\top \end{bmatrix} \in \mathbb{R}^{N \times d},$$

where d is the model dimension.

The input representations are subsequently processed by a stack of L Transformer blocks. If the transformation performed by block ℓ is denoted by $F^{(\ell)}$, then

$$X^{(\ell)} = F^{(\ell)}(X^{(\ell-1)}), \quad \ell = 1, \dots, L. \quad (58)$$

The complete Transformer stack therefore computes

$$X^{(L)} = F^{(L)} \circ F^{(L-1)} \circ \dots \circ F^{(1)}(X^{(0)}),$$

with

$$X^{(L)} \in \mathbb{R}^{N \times d}.$$

Each row of $X^{(L)}$ is the final contextualised representation associated with one token position.

For autoregressive text generation, the representation associated with the final position,

$$\mathbf{h}_N = X_{N,:}^{(L)},$$

contains the information used to predict the next token.

The language modelling head applies a linear transformation from the model dimension to the vocabulary dimension,

$$\mathbf{z} = W_{\text{LM}}\mathbf{h}_N + \mathbf{b}_{\text{LM}}, \quad (59)$$

where

$$W_{\text{LM}} \in \mathbb{R}^{|\mathcal{V}| \times d}$$

and

$$\mathbf{z} \in \mathbb{R}^{|\mathcal{V}|}.$$

The components of $\mathbf{z}$ are called **logits**. There is one logit for every token in the vocabulary.

The logits can be transformed into a probability distribution using the softmax function,

$$P(t_{N+1} = v_j \mid t_1, \dots, t_N) = \frac{\exp(z_j)}{\sum_{k=1}^{|\mathcal{V}|} \exp(z_k)}, \quad j = 1, \dots, |\mathcal{V}|. \quad (60)$$

The complete forward pass for next-token prediction can therefore be represented as

The complete forward pass for next-token prediction can therefore be represented as

$$\begin{aligned} \text{token IDs} &\longrightarrow \text{embeddings} \longrightarrow \text{Transformer blocks} \longrightarrow \mathbf{h}_N \\ &\longrightarrow \text{LM head} \longrightarrow \text{logits} \longrightarrow \text{probabilities.} \end{aligned}$$

15.2 Decoding the Next Token

After the forward pass, the language model produces a vector of logits

$$\mathbf{z} = (z_1, z_2, \dots, z_{|\mathcal{V}|}) \in \mathbb{R}^{|\mathcal{V}|},$$

with one value associated with each token in the vocabulary.

Applying softmax transforms these logits into a probability distribution,

$$p_j = \frac{\exp(z_j)}{\sum_{k=1}^{|V|} \exp(z_k)}, \quad j = 1, \dots, |V|.$$

The resulting vector

$$\mathbf{p} = (p_1, p_2, \dots, p_{|V|})$$

satisfies

$$p_j \geq 0$$

and

$$\sum_{j=1}^{|V|} p_j = 1.$$

A **decoding strategy** determines how a single token is selected from this distribution.

15.2.1 Greedy Decoding

The simplest strategy is to select the token with the highest probability,

$$j^* = \operatorname{argmax}_j p_j,$$

and therefore

$$t_{N+1} = v_{j^*}, \tag{61}$$

where v_{j^*} is the corresponding token in the vocabulary.

Because softmax preserves the ordering of the logits,

$$\operatorname{argmax}_j p_j = \operatorname{argmax}_j z_j,$$

the softmax operation does not need to be computed explicitly when greedy decoding is used.

Greedy decoding is deterministic: for a fixed model and input sequence, the same next token is selected whenever the same logits are produced.

15.2.2 Sampling

An alternative is to treat the probabilities as a categorical distribution and sample the next token according to

$$j \sim \text{Categorical}(p_1, p_2, \dots, p_{|\mathcal{V}|}). \quad (62)$$

The generated token is then

$$t_{N+1} = v_j.$$

Unlike greedy decoding, sampling is stochastic. Tokens with lower probabilities can still be selected, although tokens assigned higher probabilities by the model are more likely to be generated.

Once the next token has been selected, it is appended to the existing sequence and the autoregressive generation process continues.

16 Parallel Token Processing

One of the fundamental properties of the Transformer architecture is that the token positions within an input sequence can be processed in parallel.

Consider an input sequence containing N tokens,

$$S_N = (t_1, t_2, \dots, t_N),$$

with corresponding initial representations

$$X^{(0)} = \begin{bmatrix} \mathbf{e}_1^\top \\ \mathbf{e}_2^\top \\ \vdots \\ \mathbf{e}_N^\top \end{bmatrix} \in \mathbb{R}^{N \times d}.$$

Rather than processing these representations sequentially through a recurrent state, a Transformer operates on the complete matrix $X^{(0)}$. Each Transformer block therefore transforms representations for all token positions,

$$X^{(\ell)} = F^{(\ell)}(X^{(\ell-1)}),$$

where

$$X^{(\ell)} \in \mathbb{R}^{N \times d}.$$

The computation associated with each position is not independent of the other positions. During self-attention, information can flow between token representations, allowing the representation of a token to incorporate information from its permitted context.

For a causal language model, the representation at position i can depend only on positions up to and including i ,

$$\mathbf{h}_i = F_i(\mathbf{e}_1, \dots, \mathbf{e}_i). \quad (63)$$

It cannot depend on future positions

$$\mathbf{e}_{i+1}, \dots, \mathbf{e}_N.$$

This causal constraint does not require the positions to be processed sequentially. The computations for all positions can still be performed simultaneously while the attention mechanism prevents each position from accessing future information.

16.1 Context Length

A Transformer can process only a bounded number of token positions in a single context. If the maximum supported sequence length is $N_{\max}$, then an input sequence must satisfy

$$N \leq N_{\max}.$$

The value $N_{\max}$ defines the model's **context length**.

The context length is measured in tokens rather than words or characters. Consequently, the amount of raw text that fits into a context depends on how the tokenizer represents that text.

For autoregressive generation, the available context contains the prompt together with the tokens generated so far. As generation proceeds,

$$(t_1, \dots, t_N) \longrightarrow (t_1, \dots, t_N, t_{N+1}) \longrightarrow \dots,$$

the sequence occupying the context grows.

Parallel processing within each forward pass should therefore be distinguished from autoregressive generation across forward passes:

within a forward pass : token positions are processed in parallel,
across generation steps : new tokens are generated sequentially.

17 Key-Value Cache

Autoregressive generation repeatedly extends the input sequence by one token. Suppose that a model has already processed

$$S_N = (t_1, t_2, \dots, t_N)$$

and generated the next token t_{N+1} . The following generation step operates on

$$S_{N+1} = (t_1, t_2, \dots, t_N, t_{N+1}).$$

A naive implementation could process the entire extended sequence again. However, most of the information associated with the previous tokens has already been computed.

Transformer attention associates each token representation with three types of vectors: **queries**, **keys**, and **values**. Their precise construction will be developed when introducing self-attention. For autoregressive inference, an important observation is that the keys and values computed for previous tokens can be reused when generating subsequent tokens.

For a particular Transformer layer, let

$$K_N = \begin{bmatrix} \mathbf{k}_1^\top \\ \mathbf{k}_2^\top \\ \vdots \\ \mathbf{k}_N^\top \end{bmatrix}$$

and

$$V_N = \begin{bmatrix} \mathbf{v}_1^\top \\ \mathbf{v}_2^\top \\ \vdots \\ \mathbf{v}_N^\top \end{bmatrix}$$

denote the keys and values associated with the first N token positions.

These matrices can be stored in a **key-value cache**, or **KV cache**. When a new token t_{N+1} is processed, the model computes the new key and value

$$\mathbf{k}_{N+1} \quad \text{and} \quad \mathbf{v}_{N+1},$$

and extends the cached matrices,

$$K_{N+1} = \begin{bmatrix} K_N \\ \mathbf{k}_{N+1}^\top \end{bmatrix}, \quad V_{N+1} = \begin{bmatrix} V_N \\ \mathbf{v}_{N+1}^\top \end{bmatrix}. \quad (64)$$

The previously computed keys and values therefore do not need to be recomputed at every generation step.

Conceptually, autoregressive inference changes from repeatedly processing the complete sequence,

$$\begin{aligned} (t_1) &\longrightarrow (t_1, t_2) \longrightarrow (t_1, t_2, t_3) \\ &\longrightarrow \cdots \longrightarrow (t_1, \dots, t_N), \end{aligned}$$

to processing each new token while retaining the attention information required from the previous tokens.

The KV cache therefore trades memory for computation: cached keys and values occupy memory, but storing them reduces redundant computation during autoregressive generation.

18 Transformer Blocks

The main computational structure of a Transformer language model is a stack of **Transformer blocks**. Each block receives a sequence of token representations and produces another sequence of representations with the same model dimension.

For block ℓ , let

$$X^{(\ell-1)} \in \mathbb{R}^{N \times d}$$

denote its input and

$$X^{(\ell)} \in \mathbb{R}^{N \times d}$$

its output. The block implements a transformation

$$F^{(\ell)} : \mathbb{R}^{N \times d} \longrightarrow \mathbb{R}^{N \times d},$$

such that

$$X^{(\ell)} = F^{(\ell)}(X^{(\ell-1)}).$$

Although Transformer architectures differ in their precise implementation, a Transformer block contains two fundamental computational components:

1. a **self-attention mechanism**, which allows each token representation to incorporate information from other permitted positions in the sequence;
2. a **feedforward network**, which independently transforms the representation associated with each token position.

These components perform complementary operations. Self-attention communicates information **between token positions**, whereas the feedforward network processes the resulting information **within each token position**.

For a token representation $\mathbf{x}_i$, the distinction can be expressed schematically as

$$\begin{aligned} \text{self-attention} &: (\mathbf{x}_1, \dots, \mathbf{x}_N) \longrightarrow \mathbf{a}_i, \\ \text{feedforward network} &: \mathbf{a}_i \longrightarrow \mathbf{f}_i. \end{aligned}$$

A Transformer block also includes residual connections and normalisation operations around these transformations. Consequently, the output of a block is not obtained by simply composing attention and a feedforward network. These additional architectural components help information propagate through a deep stack of Transformer blocks.

Repeated application of the blocks progressively transforms the initial token embeddings into contextualised representations,

$$X^{(0)} \longrightarrow X^{(1)} \longrightarrow X^{(2)} \longrightarrow \dots \longrightarrow X^{(L)}.$$

The final representations $X^{(L)}$ can then be used by the language modelling head to produce logits over the vocabulary.

18.1 Feedforward Neural Network

The **feedforward neural network** (FFN) transforms each token representation independently after information has been exchanged between token positions through self-attention.

Given a token representation

$$\mathbf{x} \in \mathbb{R}^d,$$

a basic Transformer feedforward network can be written as

$$\text{FFN}(\mathbf{x}) = W_2 \phi(W_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2, \quad (65)$$

where

$$W_1 \in \mathbb{R}^{d_{\text{ff}} \times d}, \quad \mathbf{b}_1 \in \mathbb{R}^{d_{\text{ff}}},$$

and

$$W_2 \in \mathbb{R}^{d \times d_{\text{ff}}}, \quad \mathbf{b}_2 \in \mathbb{R}^d.$$

The first linear transformation projects the representation from the model dimension d into an intermediate dimension d_{ff} ,

$$\mathbb{R}^d \longrightarrow \mathbb{R}^{d_{\text{ff}}}.$$

A nonlinear activation function ϕ is then applied, followed by a second linear transformation that projects the result back to the model dimension,

$$\mathbb{R}^{d_{\text{ff}}} \longrightarrow \mathbb{R}^d.$$

The complete transformation therefore follows the dimensional structure

$$\mathbb{R}^d \longrightarrow \mathbb{R}^{d_{\text{ff}}} \longrightarrow \mathbb{R}^d.$$

The same feedforward network is applied independently to every token position. For an input matrix

$$X = \begin{bmatrix} \mathbf{x}_1^\top \\ \mathbf{x}_2^\top \\ \vdots \\ \mathbf{x}_N^\top \end{bmatrix} \in \mathbb{R}^{N \times d},$$

the operation can therefore be expressed as

$$\text{FFN}(X) = \begin{bmatrix} \text{FFN}(\mathbf{x}_1)^\top \\ \text{FFN}(\mathbf{x}_2)^\top \\ \vdots \\ \text{FFN}(\mathbf{x}_N)^\top \end{bmatrix} \in \mathbb{R}^{N \times d}.$$

No information is exchanged directly between token positions during this operation. Instead, the FFN performs a nonlinear transformation of the contextual information already contained in each token representation.

The feedforward network and self-attention therefore play complementary roles within a Transformer block: self-attention combines information across token positions, while the feedforward network transforms the representation at each position.

19 Self-Attention

The representation of a token should generally depend not only on the token itself but also on the context in which it appears. The **self-attention mechanism** allows a Transformer to construct such contextualised representations by determining which positions in the same sequence contain relevant information and combining information from those positions.

Consider a sequence of token representations

$$X = \begin{bmatrix} \mathbf{x}_1^\top \\ \mathbf{x}_2^\top \\ \vdots \\ \mathbf{x}_N^\top \end{bmatrix} \in \mathbb{R}^{N \times d}.$$

For a particular position i , self-attention constructs an output representation as a weighted combination of information associated with the available token positions,

$$\mathbf{a}_i = \sum_{j=1}^N \alpha_{ij} \mathbf{v}_j, \quad (66)$$

where $\mathbf{v}_j$ represents information obtained from position j and α_{ij} measures the relevance of that information when constructing the representation at position i .

The attention weights satisfy

$$\alpha_{ij} \geq 0$$

and

$$\sum_{j=1}^N \alpha_{ij} = 1.$$

Consequently, the output $\mathbf{a}_i$ is a weighted combination of the information available across token positions.

Conceptually, self-attention performs two operations:

1. **relevance scoring:** determine how relevant each available position j is to the position i being processed;
2. **information aggregation:** combine information from those positions according to their relevance.

The weights α_{ij} are therefore not fixed properties of the tokens. They are computed dynamically from their representations and depend on the context in which the tokens occur.

In an autoregressive language model, not every position is available to every other position. When constructing the representation at position i , the model may attend only to the current and previous positions,

$$j \leq i.$$

Therefore,

$$\alpha_{ij} = 0 \quad \text{for} \quad j > i. \quad (67)$$

This restriction prevents information from future tokens from influencing the representation used to predict the next token.

19.1 Queries, Keys, and Values

Self-attention does not compare the input token representations directly. Instead, each representation is projected into three different vector spaces, producing a **query**, a **key**, and a **value**.

Given the matrix of token representations

$$X \in \mathbb{R}^{N \times d},$$

three learned projection matrices are introduced,

$$W_Q \in \mathbb{R}^{d \times d_k}, \quad W_K \in \mathbb{R}^{d \times d_k}, \quad W_V \in \mathbb{R}^{d \times d_v}.$$

The corresponding queries, keys, and values are

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad (68)$$

with

$$Q, K \in \mathbb{R}^{N \times d_k}$$

and

$$V \in \mathbb{R}^{N \times d_v}.$$

For an individual token representation $\mathbf{x}_i$, these projections produce

$$\mathbf{q}_i = W_Q^\top \mathbf{x}_i, \quad \mathbf{k}_i = W_K^\top \mathbf{x}_i, \quad \mathbf{v}_i = W_V^\top \mathbf{x}_i.$$

The three vectors serve different purposes within the attention mechanism. The query $\mathbf{q}_i$ represents the information that position i is seeking, the key $\mathbf{k}_j$ represents the information associated with position j for relevance comparison, and the value $\mathbf{v}_j$ contains the information that position j can contribute to the resulting representation.

To determine how relevant position j is to position i , the query at position i is compared with the key at position j using their dot product,

$$s_{ij} = \mathbf{q}_i^\top \mathbf{k}_j. \quad (69)$$

A larger dot product indicates greater alignment between the query and key and therefore a larger relevance score.

For all token positions simultaneously, these scores can be computed through matrix multiplication,

$$S = QK^\top, \quad (70)$$

where

$$S \in \mathbb{R}^{N \times N}.$$

Each element s_{ij} measures the relevance of position j when constructing the contextualised representation at position i .

The attention mechanism therefore separates two operations:

$QK^\top$: determine where relevant information is located,

V : provide the information to be combined.

The score matrix S does not yet contain attention weights. The scores must first be scaled, constrained when necessary by an attention mask, and normalised before they can be used to combine the value vectors.

19.2 Scaled Dot-Product Attention

Given the query and key matrices

$$Q, K \in \mathbb{R}^{N \times d_k},$$

the dot products between every query and every key produce the score matrix

$$S = QK^\top \in \mathbb{R}^{N \times N}.$$

Each element of this matrix is

$$s_{ij} = \mathbf{q}_i^\top \mathbf{k}_j = \sum_{r=1}^{d_k} q_{ir} k_{jr}. \quad (71)$$

These scores measure the alignment between queries and keys, but their magnitude tends to increase with the dimensionality d_k . For this reason, scaled dot-product attention divides the scores by $\sqrt{d_k}$,

$$\tilde{S} = \frac{QK^\top}{\sqrt{d_k}}. \quad (72)$$

A row-wise softmax then transforms the scaled scores into attention weights,

$$A = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right), \quad (73)$$

where

$$A \in \mathbb{R}^{N \times N}.$$

For each row i ,

$$\alpha_{ij} = \frac{\exp(s_{ij}/\sqrt{d_k})}{\sum_{m=1}^N \exp(s_{im}/\sqrt{d_k})}.$$

Consequently,

$$\alpha_{ij} \geq 0$$

and

$$\sum_{j=1}^N \alpha_{ij} = 1.$$

The attention weights are then used to combine the value vectors,

$$O = AV,$$

where

$$V \in \mathbb{R}^{N \times d_v}$$

and therefore

$$O \in \mathbb{R}^{N \times d_v}.$$

Combining these operations gives the **scaled dot-product attention** function,

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V. \quad (74)$$

For an individual position i , the same operation can be written as

$$\mathbf{o}_i = \sum_{j=1}^N \alpha_{ij} \mathbf{v}_j.$$

Thus, queries and keys determine how information is weighted, while values determine what information is aggregated.

19.3 Causal Masking

In an autoregressive language model, the representation at position i must not depend on tokens appearing at future positions $j > i$. Otherwise, the model would have access to information that should not yet be available when predicting the next token.

This restriction is imposed through a **causal mask** applied to the attention scores before the softmax operation.

For a sequence of N tokens, define the mask matrix

$$M \in \mathbb{R}^{N \times N}$$

with elements

$$M_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases} \quad (75)$$

The scaled attention scores are then modified according to

$$\tilde{S} = \frac{QK^\top}{\sqrt{d_k}} + M. \quad (76)$$

For example, when $N = 4$, the causal mask has the form

$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

The first position can therefore attend only to itself, the second position can attend to the first two positions, and so forth.

Applying softmax row-wise gives the attention matrix

$$A = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right). \quad (77)$$

For a masked position $j > i$, the corresponding score is

$$\frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{d_k}} + M_{ij} = -\infty.$$

Since

$$\exp(-\infty) = 0,$$

its attention weight becomes

$$\alpha_{ij} = 0, \quad j > i.$$

Consequently, the output representation at position i depends only on the current and previous positions,

$$\mathbf{o}_i = \sum_{j=1}^i \alpha_{ij} \mathbf{v}_j. \quad (78)$$

The complete **causal scaled dot-product attention** operation can therefore be written as

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) V. \quad (79)$$

Causal masking preserves parallel processing within a forward pass while enforcing the autoregressive dependency required for next-token prediction.

19.4 Algorithm

Causal scaled dot-product self-attention can be constructed directly from the input token representations and the learned query, key, and value projection matrices.

Input: Token representation matrix $X \in \mathbb{R}^{N \times d}$, projection matrices $W_Q, W_K \in \mathbb{R}^{d \times d_k}$ and $W_V \in \mathbb{R}^{d \times d_v}$.

Output: Contextualised representation matrix $O \in \mathbb{R}^{N \times d_v}$.

1. Compute the query, key, and value projections:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

2. Compute the query-key similarity scores:

$$S = QK^\top.$$

3. Scale the scores by the square root of the key dimension:

$$\tilde{S} = \frac{S}{\sqrt{d_k}}.$$

4. Construct the causal mask

$$M_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i, \end{cases}$$

and apply it to the scaled scores:

$$\tilde{S}_{\text{masked}} = \tilde{S} + M.$$

5. Apply softmax independently to each row:

$$A = \text{softmax} \left(\tilde{S}_{\text{masked}} \right).$$

6. Combine the value vectors according to the attention weights:

$$O = AV.$$

The resulting matrix O contains one contextualised representation for each token position.

19.5 Multi-Head Attention

A single attention head computes one set of query, key, and value projections. **Multi-head attention** extends this mechanism by computing several attention heads in parallel, each with its own learned projection matrices.

Let H denote the number of attention heads. For head h , define

$$W_Q^{(h)} \in \mathbb{R}^{d \times d_k}, \quad W_K^{(h)} \in \mathbb{R}^{d \times d_k}, \quad W_V^{(h)} \in \mathbb{R}^{d \times d_v}.$$

Given

$$X \in \mathbb{R}^{N \times d},$$

each head computes

$$Q^{(h)} = XW_Q^{(h)}, \quad K^{(h)} = XW_K^{(h)}, \quad V^{(h)} = XW_V^{(h)}.$$

For causal self-attention, the output of head h is

$$O^{(h)} = \text{softmax} \left(\frac{Q^{(h)} K^{(h)\top}}{\sqrt{d_k}} + M \right) V^{(h)}, \quad (80)$$

where

$$O^{(h)} \in \mathbb{R}^{N \times d_v}.$$

The outputs of all attention heads are concatenated along the representation dimension,

$$O_{\text{cat}} = \text{Concat} \left(O^{(1)}, O^{(2)}, \dots, O^{(H)} \right),$$

giving

$$O_{\text{cat}} \in \mathbb{R}^{N \times H d_v}.$$

A final learned output projection

$$W_O \in \mathbb{R}^{H d_v \times d}$$

maps the concatenated representation back to the model dimension,

$$O = O_{\text{cat}} W_O, \tag{81}$$

where

$$O \in \mathbb{R}^{N \times d}.$$

The complete multi-head attention operation can therefore be written as

$$\text{MHA}(X) = \text{Concat} \left(O^{(1)}, \dots, O^{(H)} \right) W_O. \tag{82}$$

Because each head has its own learned projections, different heads can construct different query-key relationships and different value representations from the same input sequence. Multi-head attention therefore provides multiple learned attention mechanisms within the same Transformer block.

19.6 Multi-Query and Grouped-Query Attention

Standard **multi-head attention** assigns independent query, key, and value projections to every attention head. For H heads,

$$Q^{(h)} = XW_Q^{(h)}, \quad K^{(h)} = XW_K^{(h)}, \quad V^{(h)} = XW_V^{(h)},$$

for

$$h = 1, \dots, H.$$

Consequently, each head maintains its own keys and values.

During autoregressive inference, these keys and values are stored in the KV cache. As the context length increases, maintaining separate keys and values for every attention head increases the memory required by the cache.

19.6.1 Multi-Query Attention

Multi-query attention (MQA) retains multiple query heads but shares a single set of keys and values across them.

The query projections remain head-specific,

$$Q^{(h)} = XW_Q^{(h)}, \quad h = 1, \dots, H,$$

while the keys and values are shared,

$$K = XW_K, \quad V = XW_V.$$

Each query head therefore computes attention using the same keys and values,

$$O^{(h)} = \text{softmax} \left(\frac{Q^{(h)} K^\top}{\sqrt{d_k}} + M \right) V. \quad (83)$$

Sharing keys and values reduces the amount of information that must be stored in the KV cache during autoregressive generation.

19.6.2 Grouped-Query Attention

Grouped-query attention (GQA) provides an intermediate structure between multi-head attention and multi-query attention.

The H query heads are divided into G groups, where

$$1 < G < H.$$

Each query head retains its own query projection, while the query heads belonging to the same group share a set of keys and values.

If $g(h)$ denotes the group associated with query head h , then

$$Q^{(h)} = XW_Q^{(h)},$$

while

$$K^{(g)} = XW_K^{(g)}, \quad V^{(g)} = XW_V^{(g)}.$$

The output of query head h is therefore

$$O^{(h)} = \text{softmax} \left(\frac{Q^{(h)} K^{(g(h))^\top}}{\sqrt{d_k}} + M \right) V^{(g(h))}. \quad (84)$$

The three architectures can therefore be distinguished by how keys and values are assigned to query heads:

MHA : one key-value set per query head,

GQA : one key-value set per group of query heads,

MQA : one key-value set shared by all query heads.

GQA therefore lies between MHA and MQA, reducing the number of distinct keys and values while retaining more key-value groups than MQA.

19.7 Efficient Attention

The computational and memory requirements of attention become increasingly important as the sequence length grows. For a sequence containing N tokens, the attention score matrix

$$QK^\top \in \mathbb{R}^{N \times N}$$

contains N^2 query-key interactions.

Consequently, standard attention has quadratic complexity with respect to sequence length for the computation of these pairwise scores,

$$O(N^2).$$

Different approaches improve attention efficiency at different levels.

Sparse and local attention modify which token positions are allowed to interact. Instead of computing attention between every permitted pair of positions, they restrict attention to a subset of the sequence. This can reduce the number of query-key interactions, particularly for long sequences.

Multi-query attention and **grouped-query attention** reduce the number of distinct key and value representations by sharing them across query heads. Their main benefit during autoregressive inference is a reduction in the memory and memory bandwidth associated with the KV cache.

FlashAttention, in contrast, does not fundamentally change the mathematical definition of scaled dot-product attention. It reorganises the computation to reduce expensive data movement between different levels of GPU memory.

The conceptual attention operation remains

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) V,$$

while the implementation is arranged so that intermediate computations can make more efficient use of fast on-chip memory.

Attention efficiency can therefore be improved at different levels: by changing which positions interact, by sharing key-value representations, or by implementing the same mathematical operation more efficiently on the underlying hardware.

19.8 Residual Connections and Normalisation

Transformer blocks contain additional operations around self-attention and the feedforward network that help information propagate through a deep stack of layers. Two particularly important components are **residual connections** and **normalisation**.

19.8.1 Residual Connections

A residual connection adds the input of a transformation directly to its output. Given a representation

$$X \in \mathbb{R}^{N \times d}$$

and a transformation F with the same output shape, the residual operation is

$$Y = X + F(X). \quad (85)$$

Rather than requiring the transformation F to construct a completely new representation, the block learns a modification to the existing representation.

The dimensional compatibility

$$X, F(X) \in \mathbb{R}^{N \times d}$$

allows the two matrices to be added element-wise.

Residual connections also provide direct computational paths through the network. In a deep stack of Transformer blocks, these paths facilitate the propagation of information and gradients across layers.

19.8.2 Normalisation

Normalisation controls the scale of token representations as they pass through the network. A common operation in modern language models is **root mean square normalisation** (RMSNorm).

For a token representation

$$\mathbf{x} = (x_1, x_2, \dots, x_d) \in \mathbb{R}^d,$$

its root mean square is

$$\text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{j=1}^d x_j^2 + \varepsilon},$$

where $\varepsilon > 0$ is a small constant introduced for numerical stability.

RMSNorm transforms the representation according to

$$\text{RMSNorm}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})}, \quad (86)$$

where

$$\gamma \in \mathbb{R}^d$$

is a learned scaling vector and $\odot$ denotes element-wise multiplication.

Unlike LayerNorm, RMSNorm does not subtract the mean of the representation before rescaling it.

19.8.3 Pre-Normalisation

In a **pre-normalisation** Transformer block, normalisation is applied before each main transformation.

A simplified block can be written as

$$X' = X + \text{Attention}(\text{Norm}(X)), \quad (87)$$

followed by

$$X_{\text{out}} = X' + \text{FFN}(\text{Norm}(X')). \quad (88)$$

The residual stream therefore passes through the block while attention and the feedforward network contribute learned modifications to it.

The overall structure can be represented as

$$\begin{aligned} X &\longrightarrow \text{Norm} \longrightarrow \text{Attention} \longrightarrow +X \\ &\longrightarrow \text{Norm} \longrightarrow \text{FFN} \longrightarrow +X' \longrightarrow X_{\text{out}}. \end{aligned}$$

20 Positional Information

Language is inherently ordered. The sequences

$$(t_1, t_2, t_3)$$

and

$$(t_3, t_2, t_1)$$

contain the same tokens but generally represent different linguistic structures and meanings. A language model must therefore have access not only to token identity but also to information about token position.

Self-attention, however, does not introduce an intrinsic notion of sequence order through its query-key comparisons alone. Given token representations,

$$X = \begin{bmatrix} \mathbf{x}_1^\top \\ \mathbf{x}_2^\top \\ \vdots \\ \mathbf{x}_N^\top \end{bmatrix},$$

the attention mechanism constructs

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V,$$

and compares representations through operations such as

$$QK^\top.$$

These operations depend on the representations themselves but do not, by themselves, encode whether a representation occurs at position 1, 2, or N . Positional information must therefore be incorporated into the model.

20.1 Positional Embeddings

One approach is to associate each position i with a positional vector

$$\mathbf{p}_i \in \mathbb{R}^d.$$

Given the token embedding

$$\mathbf{e}_i \in \mathbb{R}^d,$$

the initial representation can be constructed by combining token and positional information,

$$\mathbf{x}_i = \mathbf{e}_i + \mathbf{p}_i. \tag{89}$$

For the complete sequence,

$$X = E + P,$$

where

$$E, P \in \mathbb{R}^{N \times d}.$$

The resulting representation contains information about both token identity and position before it enters the Transformer blocks.

Positional vectors may be defined by a fixed mathematical rule or learned as model parameters. In either case, their purpose is to make sequence position available to the Transformer.

Modern language models may instead incorporate positional information directly into the attention mechanism. Rotary position embeddings provide an important example of this approach.

20.2 Rotary Position Embeddings

Rotary position embeddings (RoPE) incorporate positional information directly into the query and key vectors used by self-attention. Rather than adding a positional vector to each token embedding, RoPE applies position-dependent rotations to pairs of components within the query and key representations.

Consider first a two-dimensional vector

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}.$$

A rotation by an angle θ is represented by

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}. \quad (90)$$

The rotated vector is

$$\mathbf{x}' = R(\theta)\mathbf{x}.$$

To encode token position, the rotation angle is made dependent on the position index. For a position i and a frequency ω , the rotation becomes

$$R(i\omega).$$

A query and key located at positions i and j can therefore be transformed according to

$$\tilde{\mathbf{q}}_i = R(i\omega)\mathbf{q}_i,$$

and

$$\tilde{\mathbf{k}}_j = R(j\omega)\mathbf{k}_j.$$

The attention score between the rotated vectors is proportional to their dot product,

$$\tilde{\mathbf{q}}_i^\top \tilde{\mathbf{k}}_j = \mathbf{q}_i^\top R(i\omega)^\top R(j\omega)\mathbf{k}_j.$$

Rotation matrices satisfy

$$R(\theta)^\top = R(-\theta)$$

and

$$R(\alpha)R(\beta) = R(\alpha + \beta).$$

Consequently,

$$R(i\omega)^\top R(j\omega) = R(-i\omega)R(j\omega) = R((j - i)\omega).$$

The query-key dot product therefore becomes

$$\tilde{\mathbf{q}}_i^\top \tilde{\mathbf{k}}_j = \mathbf{q}_i^\top R((j - i)\omega)\mathbf{k}_j. \tag{91}$$

The interaction between the rotated query and key thus depends naturally on the relative displacement

$$j - i.$$

For higher-dimensional query and key vectors, RoPE applies this idea to pairs of vector components using multiple rotation frequencies. The resulting position-dependent transformations can be represented schematically as

$$\tilde{Q} = \text{RoPE}(Q), \quad \tilde{K} = \text{RoPE}(K).$$

The attention operation then becomes

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{\tilde{Q}\tilde{K}^\top}{\sqrt{d_k}} + M \right) V. \quad (92)$$

RoPE therefore introduces positional information directly into the query-key interactions that determine attention weights, while the value vectors are not rotated by the positional transformation.

21 The Transformer Decoder

A decoder-only language model combines the components developed in the previous sections into a sequence of Transformer decoder blocks. These blocks progressively transform token representations while preserving the model dimension.

Given a sequence of token IDs

$$(i_1, i_2, \dots, i_N),$$

the embedding layer produces the initial representation matrix

$$X^{(0)} \in \mathbb{R}^{N \times d}.$$

This matrix is passed through a stack of L Transformer blocks,

$$X^{(0)} \longrightarrow X^{(1)} \longrightarrow \dots \longrightarrow X^{(L)}.$$

For a pre-normalisation decoder block, the attention sublayer can be represented schematically as

$$U^{(\ell)} = X^{(\ell-1)} + \text{Attention}^{(\ell)} \left(\text{Norm}_1^{(\ell)} (X^{(\ell-1)}) \right). \quad (93)$$

The feedforward sublayer then computes

$$X^{(\ell)} = U^{(\ell)} + \text{FFN}^{(\ell)} \left(\text{Norm}_2^{(\ell)} (U^{(\ell)}) \right). \quad (94)$$

Within the attention sublayer, the normalised representations are projected into queries, keys, and values. Positional information can be incorporated through rotary position embeddings applied to the queries and keys,

$$Q \rightarrow \tilde{Q}, \quad K \rightarrow \tilde{K}.$$

Causal self-attention then computes

$$\text{Attention}(\tilde{Q}, \tilde{K}, V) = \text{softmax}\left(\frac{\tilde{Q}\tilde{K}^\top}{\sqrt{d_k}} + M\right)V.$$

Multiple attention heads perform this operation in parallel, and their outputs are combined and projected back to the model dimension before entering the residual stream.

After the final Transformer block, the resulting representations have the form

$$X^{(L)} = \begin{bmatrix} \mathbf{h}_1^\top \\ \mathbf{h}_2^\top \\ \vdots \\ \mathbf{h}_N^\top \end{bmatrix} \in \mathbb{R}^{N \times d}.$$

For next-token generation, the final representation at the last position,

$$\mathbf{h}_N \in \mathbb{R}^d,$$

is mapped by the language modelling head to vocabulary logits,

$$\mathbf{z}_N = W_{\text{LM}}\mathbf{h}_N + \mathbf{b}_{\text{LM}},$$

where

$$\mathbf{z}_N \in \mathbb{R}^{|\mathcal{V}|}.$$

A decoding strategy selects the next token from these scores. The selected token is appended to the sequence, and the process is repeated autoregressively.

The complete decoder-only generation pipeline can therefore be summarised as

$$\begin{aligned} \text{tokens} &\rightarrow \text{embeddings} \rightarrow \text{Transformer blocks} \\ &\rightarrow \text{LM head} \rightarrow \text{logits} \rightarrow \text{decoding}. \end{aligned}$$

22 Inspecting a Transformer Model

The internal structure of a pretrained language model can be inspected directly using PyTorch and Hugging Face Transformers. This provides a connection between the mathematical description of a Transformer decoder and its software implementation.

The following example uses the same pretrained model employed in the previous examples.

The inspection focuses on the main architectural components and their dimensions rather than on the numerical values of the model parameters.

In particular, the model allows us to identify the correspondence

$$\begin{aligned}\text{embedding layer} &\longleftrightarrow X^{(0)}, \\ \text{decoder layers} &\longleftrightarrow F^{(1)}, \dots, F^{(L)}, \\ \text{attention projections} &\longleftrightarrow W_Q, W_K, W_V, \\ \text{feedforward layers} &\longleftrightarrow \text{FFN}, \\ \text{normalisation} &\longleftrightarrow \text{Norm}, \\ \text{language modelling head} &\longleftrightarrow W_{\text{LM}}.\end{aligned}$$

Inspecting the corresponding tensor dimensions also provides a practical way to verify how the model dimension, attention heads, key-value heads, feedforward dimension, and vocabulary size determine the architecture.

22.1 Model Architecture

For the inspected model,

$$|\mathcal{V}| = 49152, \quad d = 576, \quad L = 30,$$

with

$$H = 9$$

attention heads and

$$H_{\text{KV}} = 3$$

key-value heads.

The dimension of each attention head is therefore

$$d_k = \frac{d}{H} = \frac{576}{9} = 64.$$

The query projection produces

$$9 \times 64 = 576$$

components, while the key and value projections each produce

$$3 \times 64 = 192$$

components. The model therefore uses grouped-query attention, with three query heads sharing each key-value head.

The attention dimensions can be summarised as

$$\begin{aligned} Q &\in \mathbb{R}^{N \times 9 \times 64}, \\ K &\in \mathbb{R}^{N \times 3 \times 64}, \\ V &\in \mathbb{R}^{N \times 3 \times 64}. \end{aligned}$$

After attention, the head outputs are combined into the model dimension,

$$9 \times 64 = 576,$$

and the output projection maps

$$\mathbb{R}^{576} \longrightarrow \mathbb{R}^{576}.$$

The feedforward network uses an intermediate dimension

$$d_{\text{ff}} = 1536$$

before projecting its output back to the model dimension. The block also contains two RMSNorm operations acting on representations in $\mathbb{R}^{576}$.

Finally, the language modelling head maps

$$\mathbb{R}^{576} \longrightarrow \mathbb{R}^{49152},$$

producing one logit for every token in the vocabulary.

22.1.1 Gated Feedforward Networks

Modern Transformer architectures often replace the conventional feedforward network with a **gated feedforward network**.

A standard feedforward network can be written as

$$\text{FFN}(\mathbf{x}) = W_2 \phi(W_1 \mathbf{x}),$$

where the representation is first projected to a larger intermediate dimension, transformed by a nonlinear activation function, and then projected back to the model dimension.

A gated feedforward network instead constructs two intermediate representations. A common form is

$$\text{GatedFFN}(\mathbf{x}) = W_{\text{down}} \left[\phi(W_{\text{gate}} \mathbf{x}) \odot (W_{\text{up}} \mathbf{x}) \right], \quad (95)$$

where $\odot$ denotes element-wise multiplication.

The two branches play different roles. The projection

$$W_{\text{up}} \mathbf{x}$$

produces an intermediate representation, while

$$\phi(W_{\text{gate}} \mathbf{x})$$

produces a nonlinear gate that modulates that representation element by element.

When the activation function is SiLU,

$$\text{SiLU}(x) = x \sigma(x),$$

this structure corresponds to the SwiGLU family of gated feedforward networks.

For a model dimension d and intermediate dimension d_{ff} ,

$$W_{\text{gate}}, W_{\text{up}} : \mathbb{R}^d \rightarrow \mathbb{R}^{d_{\text{ff}}},$$

while

$$W_{\text{down}} : \mathbb{R}^{d_{\text{ff}}} \rightarrow \mathbb{R}^d.$$

The feedforward sublayer therefore preserves the model dimension while using a larger internal representation and a learned gating mechanism.